CYBERSHIELD
SMART THREAT DETECTION

A Project Report

Submitted in partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

Third Year B.Sc. Computer Science

Submitted by

LAXMI YADAV

Roll No.: CS24027

PNR No.: MU0341120240166885

Under the esteemed guidance of

Asst. Prof. Swati Pillai

(Project Guide)

DEPARTMENT OF COMPUTER SCIENCE

S.S.T COLLEGE OF ARTS & COMMERCE

(Affiliated to University of Mumbai)

ULHASNAGAR, 421004

MAHARASHTRA

2026 - 2027

S.S.T COLLEGE OF ARTS & COMMERCE

(Affiliated to University of Mumbai)

ULHASNAGAR - MAHARASHTRA - 421004

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled **"CyberShield - Smart Threat Detection"** is submitted by **LAXMI YADAV**, bearing Roll No. **CS24027** and PNR No. **MU0341120240166885**, in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** from the University of Mumbai for the academic year **2026 - 2027**.

The project report has been prepared under the guidance of **Asst. Prof. Swati Pillai** in the Department of Computer Science, S.S.T College of Arts & Commerce.

_____ _____

Internal Guide Coordinator

_____

External Examiner

Date: _____ College Seal

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

PNR No.: MU0341120240166885

Roll No.: CS24027

1. Name of the Student: Laxmi Yadav

2. Title of the Project: CyberShield - Smart Threat Detection

3. Name of the Guide: Asst. Prof. Swati Pillai

4. Class: Third Year B.Sc. Computer Science

5. Academic Year: 2026 - 2027

6. Teaching experience of the Guide: _____

7. Is this your first submission?: Yes [ ] No [ ]

_____ _____

Signature of the Student Signature of the Guide

Date: _____ Date: _____

_____

Signature of the Coordinator

ABSTRACT

CyberShield - Smart Threat Detection is a browser-based cyber-security application designed to help everyday users assess suspicious links, downloaded files, SMS or email messages, QR codes, IP addresses and password exposure. Fraud often combines several channels: a threatening SMS may contain a phishing link, a QR code may conceal a payment request, and an unfamiliar file may be shared as an attachment. CyberShield brings these checks together in a single interface and explains the available findings in plain language.

The frontend is developed using React, Vite, Tailwind CSS and Framer Motion. A Node.js and Express API coordinates Google Safe Browsing, VirusTotal, RDAP, AbuseIPDB, ip-api and Gemini. Browser-side Web Crypto supports file hashing and password privacy; jsQR decodes uploaded images or camera frames. Password exposure is checked through the Have I Been Pwned hash-range API, which receives only a five-character SHA-1 prefix.

Deterministic scoring functions combine reputation results and structural indicators into a bounded score from 0 to 100. The verdict bands are Safe, Suspicious and Dangerous; unavailable file or IP reputation produces an unscored result. Gemini provides message analysis or short explanations, with rule-based fallback when the service is unavailable. Source availability and a score breakdown make the result understandable without representing a score as a guarantee of safety.

The latest report is retained in browser session storage and can be downloaded as a PDF. Firestore stores the public total-scan counter. The frontend is deployed on GitHub Pages and the API on a Render Web Service. Verification includes 22 automated tests, targeted live regression checks and 26 normal-navigation checks. The results document functional correctness for the executed test cases.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Asst. Prof. Swati Pillai, for her guidance, encouragement and support in the preparation of this project report. Her academic direction helped me organise the work and present the design, implementation and testing of CyberShield in a systematic manner.

I am thankful to the Department of Computer Science and S.S.T College of Arts & Commerce, affiliated to the University of Mumbai, for providing an academic environment in which I could study web development and practical cyber-security concepts. This report is prepared as part of the Third Year B.Sc. Computer Science curriculum.

I acknowledge the official documentation and openly available developer resources of React, Node.js, Express, Firebase and the security-service providers used by the application. These resources supported the selection of appropriate tools, the understanding of API behaviour and the implementation of privacy-conscious input handling.

I also thank my classmates, friends and family for their encouragement and suggestions. Their support helped me approach the project patiently, review the user experience and improve the clarity of the final documentation.

Laxmi Yadav
Roll No.: CS24027
PNR No.: MU0341120240166885

DECLARATION

I, Laxmi Yadav, bearing Roll No. CS24027 and PNR No. MU0341120240166885, declare that this project report entitled "CyberShield - Smart Threat Detection" has been prepared for submission in partial fulfilment of the requirements of the Bachelor of Science (Computer Science) programme at S.S.T College of Arts & Commerce, affiliated to the University of Mumbai.

The report describes the design, implementation, deployment and testing of CyberShield. External libraries, technical documentation and security APIs are acknowledged in the references. The system combines service findings with transparent weighted scoring.

The implementation details and recorded test outcomes in this report are presented accurately to the best of my knowledge. The proposed enhancements and estimated development schedule are distinguished from implemented features and recorded test results.

This declaration is submitted for the academic year 2026 - 2027 under the guidance of Asst. Prof. Swati Pillai.

Place: Ulhasnagar Date: _____

Student signature: _____ Laxmi Yadav

INDEX

SECTION	TOPIC	PAGE
1	INTRODUCTION	11
1.1	Project overview	11
1.2	Background and motivation	12
1.3	Objectives and purpose	13
1.4	Scope, applicability and achievements	14
2	SURVEY OF TECHNOLOGIES	15
2.1	Frontend technology survey	15
2.2	Backend, intelligence and reporting	16
2.3	Comparison of alternatives	17
3	REQUIREMENTS AND ANALYSIS	18
3.1	Problem and requirement specification	18
3.2	Functional / non-functional requirements	19
3.3	Planning, scheduling and PERT	21
3.4	System analysis	23
3.5	Software and hardware requirements	24
4	SYSTEM DESIGN	25
4.1	Basic modules	26
4.2	Data design, report and integrity	27
4.3	Procedural design and diagrams	29
4.4	Interface and AI role	40
4.5	Security issues and test-case design	41

INDEX (CONTINUED)

SECTION	TOPIC	PAGE
5	IMPLEMENTATION & TESTING	42
5.1	Environment and project organisation	42
5.2	Selected code implementation	44
5.3	System testing and automated results	53
5.4	Functional, regression and navigation tests	55
5.5	Deployment and configuration	59
5.6	Provider limits and verification boundaries	60
6	RESULTS & DISCUSSION	61
6.1	Observed results and score interpretation	61
6.2	Application screenshots	63
6.3	Advantages and limitations	72
6.4	Future enhancements	73
7	CONCLUSION	74
8	REFERENCES	75

The index lists the printed page numbers for each chapter and section of the CyberShield project report.

LIST OF FIGURES

FIGURE	PAGE
Figure 3-1: Estimated eight-week development schedule	21
Figure 3-2: Iterative development and verification process	21
Figure 3-3: PERT dependency network for the development plan	22
Figure 4-1: Layered architecture of CyberShield	25
Figure 4-2: URL analysis logic flow	29
Figure 4-3: User-facing use cases	30
Figure 4-4: Component and service module relationships	31
Figure 4-5: URL scan sequence across system boundaries	32
Figure 4-6: Context data flow diagram (Level 0)	33
Figure 4-7: Scanner data flow diagram (Level 1)	33
Figure 4-8: Hash-first file analysis activity	34
Figure 4-9: Conceptual report and storage relationships	35
Figure 4-10: Representative URL risk weights	39
Figure 4-11: Provider-assisted explanation pipeline	40
Figure 5-1: Recorded automated test outcomes	54
Figure 5-2: Navigation-regression coverage	58
Figure 5-3: GitHub Pages and Render deployment flow	59
Figure 6-1: Scores of executed functional test cases	62
Figure 6-2: CyberShield Home page	63
Figure 6-3: Link Scanner form and source panel	63
Figure 6-4: File Scanner input	64
Figure 6-5: SMS / Email Scanner input	64
Figure 6-6: QR Scanner input methods	65
Figure 6-7: IP Address Checker form	65
Figure 6-8: Password Leak Checker and privacy explanation	66
Figure 6-9: Awareness page and golden rules	66
Figure 6-10: Report for the normal Google URL	67
Figure 6-11: Report for the official phishing test URL	67
Figure 6-12: Dangerous UPI refund-trap report	68
Figure 6-13: Scam SMS report and explanation	68
Figure 6-14: Unscored private-IP report	69
Figure 6-15: Breached-password report	69
Figure 6-16: Ten-question awareness quiz	70
Figure 6-17: Exported UPI report excerpt with readable currency	70
Figure 6-18: Published CyberShield repository	71
Figure 6-19: Formatted response from the deployed API health endpoint	71

LIST OF TABLES

TABLE	PAGE
Table 2-1: Selected technologies and alternatives	17
Table 3-1: Scanner functional requirements	19
Table 3-2: Non-functional requirements	20
Table 3-3: PERT effort estimates	22
Table 3-4: Existing approaches and proposed system	23
Table 3-5: Software requirements	24
Table 3-6: Suggested hardware and connectivity	24
Table 4-1: Modules and responsibilities	26
Table 4-2: Persistent and temporary data stores	27
Table 4-3: Shared report fields	28
Table 4-4: Important data structures	35
Table 4-5: URL and backend scoring weights	38
Table 4-6: Verdict bands	38
Table 4-7: Local scoring rules	39
Table 4-8: Security controls and scope	41
Table 5-1: Verified implementation environment	42
Table 5-2: Project files and their purpose	43
Table 5-3: Verification suites and their scope	53
Table 5-4: Automated tests by area	54
Table 5-5: Functional URL, file and IP cases	55
Table 5-6: Browser-local and message tests	56
Table 5-7: Confirmed defects and resulting corrections	57
Table 5-8: Published system addresses	59
Table 5-9: Relevant provider and application limits	60
Table 6-1: Recorded functional test results	61

CHAPTER 1
INTRODUCTION

CyberShield is a web application that combines multiple practical safety checks in one place. Instead of asking a user to interpret unrelated security websites, it accepts an item, gathers relevant evidence, computes an explainable threat score and presents the findings with suggested next steps. Its six scanner categories are Link, File, SMS / Email, QR Code, IP Address and Password.

The application is aimed at students and general users who receive unfamiliar links or messages, download software or documents, encounter UPI QR requests, or want to check whether a password is known in breach data. The interface does not require an account. It is designed as an educational decision-support tool that makes security information easier to understand.

A dark cyber-security theme, responsive cards, a radar-style progress overlay and colour-coded verdicts provide a consistent experience. Beneath that presentation, the project separates browser-only work from backend integrations. The browser computes file fingerprints and password hashes; the backend keeps private service keys and coordinates the network checks.

1.1 Project overview

• Six scanner modules share a common result format.

• Source-specific cards and a score breakdown explain the verdict.

• Unavailable or pending evidence is communicated instead of inventing a completed check.

• The latest browser-session report survives refresh and can be exported as PDF.

• An Awareness section explains common scams and includes a ten-question quiz.

CyberShield does not execute uploaded files, make payments, crack passwords or guarantee that an input is safe. It combines provider intelligence with local rules and presents the limits of the available evidence.

1.2 Background and motivation

Online fraud increasingly uses everyday communication channels. A message may imitate a bank and create urgency around KYC, electricity bills or account suspension. A fake job offer may demand a registration fee. A QR code may be presented as a refund mechanism even though the encoded UPI request is intended to send money. These situations require an understandable response before a user acts.

HTTPS encrypts a connection but does not establish that a website is legitimate. Attackers can register a new domain, use misleading subdomains or copy branding. Conversely, a suspicious-looking keyword alone is not sufficient proof of fraud. A useful scanner should therefore combine several forms of evidence and explain why each contributes to risk.

Existing services provide valuable but specialised information. Google Safe Browsing checks known threat lists; VirusTotal aggregates engine findings; AbuseIPDB focuses on reported IP abuse; RDAP provides domain registration data. Password-range lookups and QR decoding address different problems. CyberShield connects these capabilities through a common workflow.

1.2.1 Motivation for an integrated system

• Reduce the effort of switching between separate checking tools.

• Translate technical findings into simple explanations and practical advice.

• Preserve password privacy through a hash-prefix lookup.

• Offer local scam-pattern checks when an AI response is unavailable.

• Use transparent rules rather than an unexplained binary label.

The motivation is educational and preventive. The system helps users review evidence; it does not replace a bank, an incident-response team or independent verification through an official channel.

1.3 Objectives and purpose

1.3.1 Objectives

• Validate supported inputs before processing them.

• Check URLs using Google Safe Browsing, VirusTotal, RDAP and structural heuristics.

• Use SHA-256 hash-first file lookup and upload only an unknown file when analysis requires it.

• Analyse scam messages with Gemini and deterministic local patterns, including up to three embedded links.

• Decode QR content locally and distinguish website URLs, UPI payment requests and other data.

• Check public IP reputation and avoid assigning a public-reputation score to private or reserved addresses.

• Check password breach exposure using a five-character SHA-1 prefix and estimate local password strength.

• Display an itemised score explanation, source availability and useful next actions.

• Support PDF export, session persistence, responsive navigation and cyber-awareness learning.

1.3.2 Purpose

The purpose of the project is to demonstrate the application of modern web technologies to a real cyber-security problem. It shows how input validation, asynchronous APIs, structured data, weighted scoring, privacy controls and usability can work together in an accessible application.

A single scan does not use every source. Each module selects the relevant providers: password checks contact HIBP directly from the browser, while URL and public-IP intelligence are coordinated by the API. This separation reduces unnecessary data transfer and clarifies the privacy boundary.

1.4 Scope, applicability and achievements

1.4.1 Scope

The completed scope includes six scanners, a shared Result page, PDF reports, a public scan counter, an Awareness section and public deployment. File uploads are limited to 32 MiB; messages to 5,000 characters; URL input to 2,048 characters; and the password form to 128 characters. Up to three message links are analysed.

1.4.2 Applicability

• Students can use the project to understand phishing signals and API integration.

• General users can review suspicious communication and unfamiliar downloads.

• Cyber-awareness activities explain UPI refund traps and password breach exposure without making a payment.

• Academic evaluation can trace each module from requirements to implementation and tests.

1.4.3 Achievements

The deployed frontend and backend work together over HTTPS. Confirmed defects in QR URL classification, scan cancellation, IPv6 validation, cached file identity, keyboard navigation and PDF currency rendering were corrected. A route-animation defect that hid lazy-loaded pages was also corrected and checked with normal navigation.

Twenty-two automated tests pass in the recorded verification. Seventeen targeted deployed regression and smoke checks passed after the scanner corrections, and a later suite of 26 deployed navigation checks passed after the route-visibility correction. These are separate test suites and should not be added together as a detection-accuracy percentage.

CHAPTER 2
SURVEY OF TECHNOLOGIES

2.1 Available technologies: frontend

React is used to build reusable interface components and manage form and page state. Scanner inputs, source cards, result gauges, navigation and awareness content are organised into components rather than a single large page. React Router maps Home, Scanner, Result, Awareness and missing-page routes.

Vite provides the development server and production build. It proxies local /api requests to the Express server and supports route-level code splitting. GitHub Pages uses a /cybershield/ base path and hash routes so links such as #/scan/url can be opened and refreshed on static hosting.

Tailwind CSS defines responsive spacing, layout and colour utilities. The visual theme is implemented through shared tokens, glass-style panels and reusable buttons. Framer Motion supports component animations, while reduced-motion preferences are respected. The route shell itself renders immediately rather than waiting for an exiting page animation.

2.1.1 Browser platform

Web Crypto supplies SHA-256 for file fingerprinting and SHA-1 for the HIBP range protocol. Canvas and jsQR handle image decoding. getUserMedia provides a camera stream when the browser and permission state permit it. Browser session storage retains the latest report, and local storage records the best quiz score.

Modern browsers with HTTPS, JavaScript, Web Crypto and media-device support are appropriate. Camera and cryptographic processing require compatible browser APIs. Physical camera performance depends on the device, lighting and permission settings.

2.2 Backend, intelligence and reporting technologies

Node.js provides an event-driven JavaScript runtime for the API. Express supplies routing, JSON parsing and middleware. Axios is used by the server-side service adapters to call provider endpoints with explicit timeouts. Multer receives file uploads in memory, with a 32 MiB size cap.

Google Safe Browsing and VirusTotal provide URL/file threat evidence. RDAP.org routes domain-registration queries to the relevant registry. AbuseIPDB provides public-IP reputation; ip-api enriches location and network details. Gemini analyses message text and generates short explanations. The application uses external service integrations and deterministic scoring functions.

Have I Been Pwned offers the Pwned Passwords range API. Only a five-character hash prefix is sent, and the browser compares the returned suffixes locally. This service is separate from paid email-address breach-search APIs, which are not implemented in CyberShield.

Firebase Firestore is used for the public totalScans counter, not for a database of personal scan history. jsPDF and jsPDF-AutoTable create downloadable reports in the browser. GitHub Actions builds and deploys the static frontend; Render runs the Node Web Service.

2.2.1 Development and verification

The project uses ES modules, npm lockfiles, the Node test runner and oxlint. Browser automation was used to verify the published site and its navigation. PDF text extraction and image rendering were used to check the report export, including the corrected INR representation.

2.3 Comparison of alternatives

Table 2-1: Selected technologies and alternatives

Concern	Selected approach	Alternative / reason
Interface	React components	Static HTML is simpler, but stateful scanner/results benefit from components.
API server	Node.js + Express	Flask is viable; JavaScript on both sides shares concepts and data shapes.
Build	Vite	A manual build would require more configuration and asset management.
Threat intelligence	Multiple security APIs	A single blacklist has narrower coverage and offers fewer explanations.
Message analysis	Gemini + rules	Rules alone work offline but are less flexible with wording.
File handling	Hash-first lookup	Uploading every file increases exposure, bandwidth and provider requests.
Storage	Firestore counter + browser session	A scan-history database requires accounts, retention rules and more personal data.
Export	Client-side PDF	Server-side reporting adds another backend workload and transfer step.
Hosting	GitHub Pages + Render	A single server can host both, but needs frontend-serving configuration.

The selected design provides modular service integration and explainable results. Its principal constraints are provider quotas, service availability and interpretation of heuristic scores. Gemini is used through its API; the application does not include a local model-training stage.

The project can later migrate to persistent backend caching, authenticated history and calibrated detection thresholds. These are future extensions rather than features claimed in the present implementation.

CHAPTER 3
REQUIREMENTS AND ANALYSIS

3.1 Problem definition

Users need a convenient way to review potentially harmful digital content before opening a link, installing a file, responding to a message, scanning a payment QR or reusing an exposed password. The challenge is to gather meaningful evidence while keeping the process understandable and limiting unnecessary disclosure.

A purely binary decision hides uncertainty. Provider responses may be absent, queued, rate-limited or outdated. A long URL may contain suspicious words while belonging to a legitimate website. A private IP address has no applicable public-reputation lookup. The problem therefore includes communicating uncertainty as well as producing a score.

3.2 Requirement specification

• Accept each supported input through the appropriate scanner form.

• Keep private security-service keys outside the browser bundle.

• Use bounded requests and validate inputs before provider calls.

• Calculate reproducible score breakdowns using centrally defined weights.

• Provide consistent report fields for all modules.

• Support browser-session persistence and PDF download.

• Expose source availability and preserve unscored states where evidence is absent.

The system should remain usable when one provider fails. This means the interface must still render, local checks must continue where available, and missing findings must be visibly distinguished from a completed clean result.

3.2.1 Functional requirements

Table 3-1: Scanner functional requirements

ID	Module	Requirement
FR01	Link	Accept HTTP(S) URLs and bare domains; normalise input and query relevant sources.
FR02	File	Compute SHA-256 in the browser, look up the hash, and upload only when unknown.
FR03	SMS / Email	Accept text, apply AI/rules, highlight red flags and scan at most three links.
FR04	QR Code	Decode a camera/image/text input and classify URL, UPI, Wi-Fi, phone or plain content.
FR05	IP Address	Validate IPv4/IPv6, normalise equivalent spellings and exclude non-public ranges.
FR06	Password	Perform HIBP prefix lookup and local strength estimation without saving the raw password.
FR07	Result	Show score, verdict, explanation, breakdown, per-source findings and advice.
FR08	Persistence	Retain only the latest report in browser session storage; keep the quiz best score locally.
FR09	Export	Generate a readable PDF with the scored input, findings and timestamp.
FR10	Awareness	Display seven scam explainers and a ten-question multiple-choice quiz.
FR11	Counter	Read a live public Firestore counter and increment after a completed scan.

For URL-based QR codes, the original query string must be preserved and the destination sent through the Link Scanner. A parameter such as pa= alone does not establish that a QR is a UPI payment request.

3.2.2 Non-functional requirements

Table 3-2: Non-functional requirements

Area	Required behaviour
Usability	Clear labels, actionable errors, simple explanations and a consistent scanner/result workflow.
Availability	Continue with local evidence or a partial result when a provider is unavailable.
Privacy	Do not transmit raw passwords; document message/provider transfer and session retention.
Security	Validate schemes, hashes and IP input; restrict CORS; limit requests and file/body sizes.
Maintainability	Separate routes, service adapters, scoring functions and UI components.
Accessibility	Keyboard navigation, visible loading/error states, reduced motion and responsive layouts.
Performance	Run independent sources in parallel and cache reusable findings.
Transparency	Explain score contributions and avoid presenting missing evidence as a completed check.
Portability	Support modern browsers and a Node runtime compatible with installed dependencies.

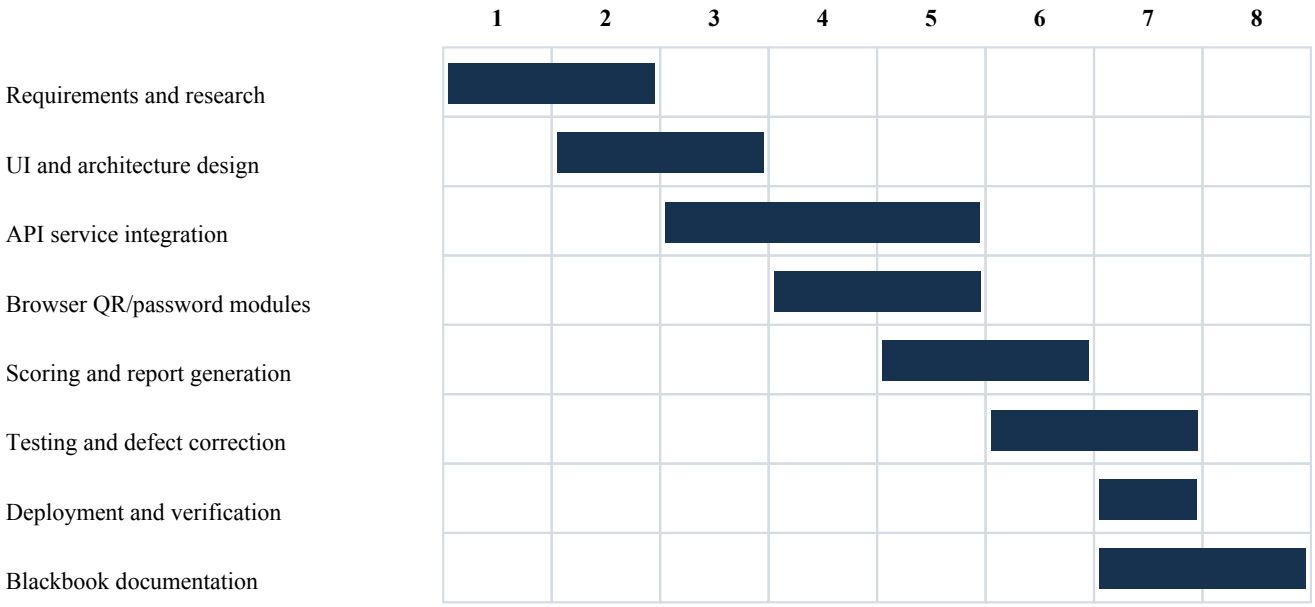
3.2.3 Constraints

Free-service quotas constrain throughput. One backend request can produce several provider calls, especially a new VirusTotal analysis. A per-user rate limiter cannot by itself guarantee the shared upstream quota. The free Render instance may sleep after inactivity; a cold first request may be slower.

Scoring weights are hand-designed for explainability. They have not been calibrated using a large independent labelled dataset. The system should therefore present the score as an indicator rather than a probability of fraud.

3.3 Planning and scheduling

The development plan covers requirements, design, implementation, integration, verification and documentation. The schedule assigns estimated time to each activity and identifies dependencies between the stages.



Week numbers denote the estimated development schedule.

Figure 3-1: Estimated eight-week development schedule

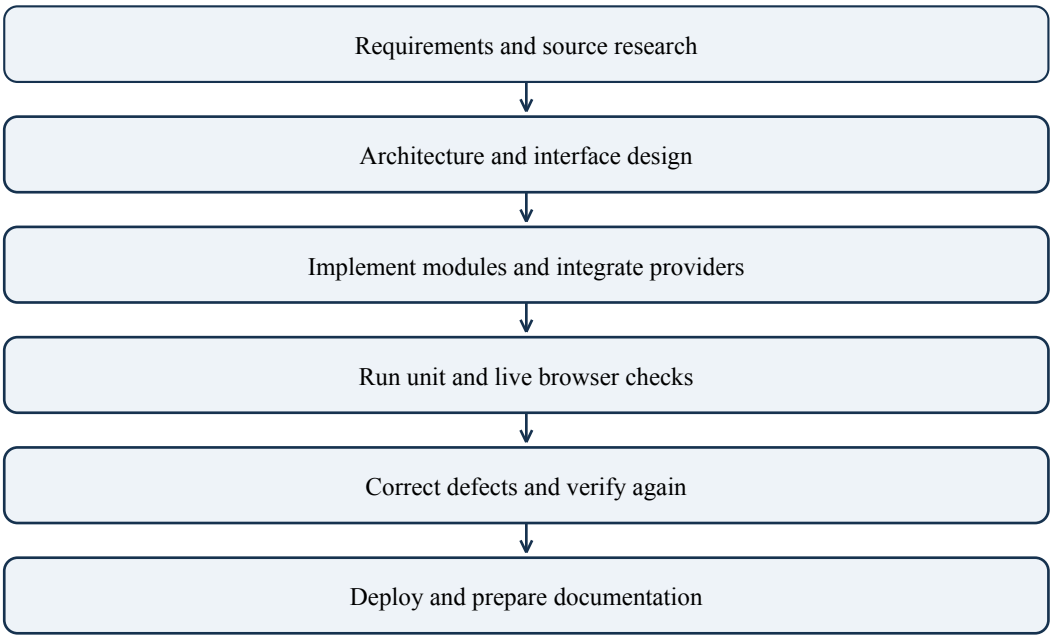


Figure 3-2: Iterative development and verification process

Verification feeds back into implementation. Normal-navigation tests were added after route-transition inspection identified a defect that refresh-based checks had not detected.

3.3.1 PERT planning and estimation

PERT expresses uncertainty using optimistic (O), most-likely (M) and pessimistic (P) effort estimates. The expected duration is E = (O + 4M + P) / 6. The task durations below are planning estimates used to determine the expected development sequence.

Table 3-3: PERT effort estimates

Task	Predecessor	O	M	P	E days
A Requirements	-	2	3	5	3.17
B Architecture	A	2	3	4	3.00
C API services	B	4	6	9	6.17
D Browser modules	B	3	4	7	4.33
E Scoring/report integration	C, D	3	5	8	5.17
F Tests and fixes	E	3	5	9	5.33
G Deployment	F	1	2	4	2.17
H Documentation	G	3	4	6	4.17

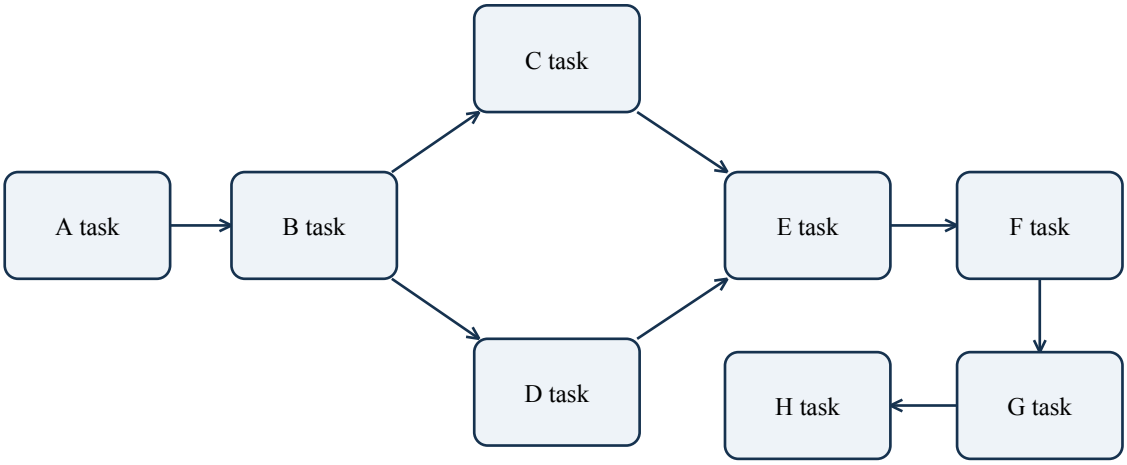


Figure 3-3: PERT dependency network for the development plan

The API-services branch has a longer estimated duration than the browser-modules branch. The critical path is A-B-C-E-F-G-H, with a total estimated duration of approximately 29.18 days. PERT supports scheduling and dependency analysis; these values do not measure application response time.

3.4 System analysis

Table 3-4: Existing approaches and proposed system

Aspect	Separate/manual approach	CyberShield
User workflow	Visit individual services and interpret outputs	Choose a scanner and receive a common report.
Evidence	Often one specialised source	Combine the relevant providers with local rules.
Explanations	Technical response or a simple label	Itemised score contributions and plain-language advice.
Password handling	Risk of sending a password to an unfamiliar site	HIBP range query sends only a hash prefix.
QR interpretation	Destination may not be obvious	Decode locally and expose the content before acting.
Unavailable data	May be mistaken for a clean check	Availability cards, partial-result notices and unscored states.
Education	External articles or ad hoc guidance	Seven scam cards and a ten-question quiz.
Reports	Manual screenshots or copying	Client-generated PDF and last-session result.

3.4.1 Feasibility

Technical feasibility is supported by browser APIs, maintained JavaScript libraries and available provider interfaces. Economic feasibility is supported by free or limited developer offerings and static frontend hosting. Operational feasibility depends on clear instructions, usable errors and reasonable expectations about delays and incomplete findings.

The project is designed for academic evaluation and cyber-security awareness. Sustained public traffic requires provider-level quota controls, stronger operational monitoring and a hosting plan selected for the intended workload.

3.5 Software and hardware requirements

Table 3-5: Software requirements

Component	Requirement / purpose
Node.js	Node 22.12+ recommended; installed Vite requires Node 20.19+ or 22.12+, Gemini SDK requires Node 20+.
npm	Install dependencies, run tests and produce builds.
React / React DOM	Component rendering and frontend state.
Vite / Tailwind / Framer Motion	Build process, responsive styling and interface animation.
Express / Axios / Multer	API routes, provider requests and memory-only uploads.
Firebase SDK	Optional Firestore public-counter access.
jsQR / Web Crypto / jsPDF	QR decode, browser hashing and PDF reports.
Browser	Current Chrome or equivalent with HTTPS, JavaScript, storage and Web Crypto.
Development tools	Editor, Git, Node test runner, oxlint and browser inspection tools.

Table 3-6: Suggested hardware and connectivity

Item	Suggested specification
Development machine	Modern 64-bit computer; 4 GB RAM minimum, 8 GB recommended.
Storage	Enough space for source, npm dependencies, build output and test artifacts.
Network	Internet access for provider checks and deployment.
Camera	Optional webcam or mobile camera for live QR reading.
Display	Responsive browser viewport; narrow layouts checked at 320, 375 and 768 px.

These hardware specifications are recommended for development, testing and project evaluation. The application can use image upload or pasted QR text when a camera is unavailable.

CHAPTER 4
SYSTEM DESIGN

CyberShield separates presentation, browser-local processing, backend orchestration and external intelligence. The result object is the shared contract between these layers. The architecture avoids placing private provider credentials in the frontend bundle.

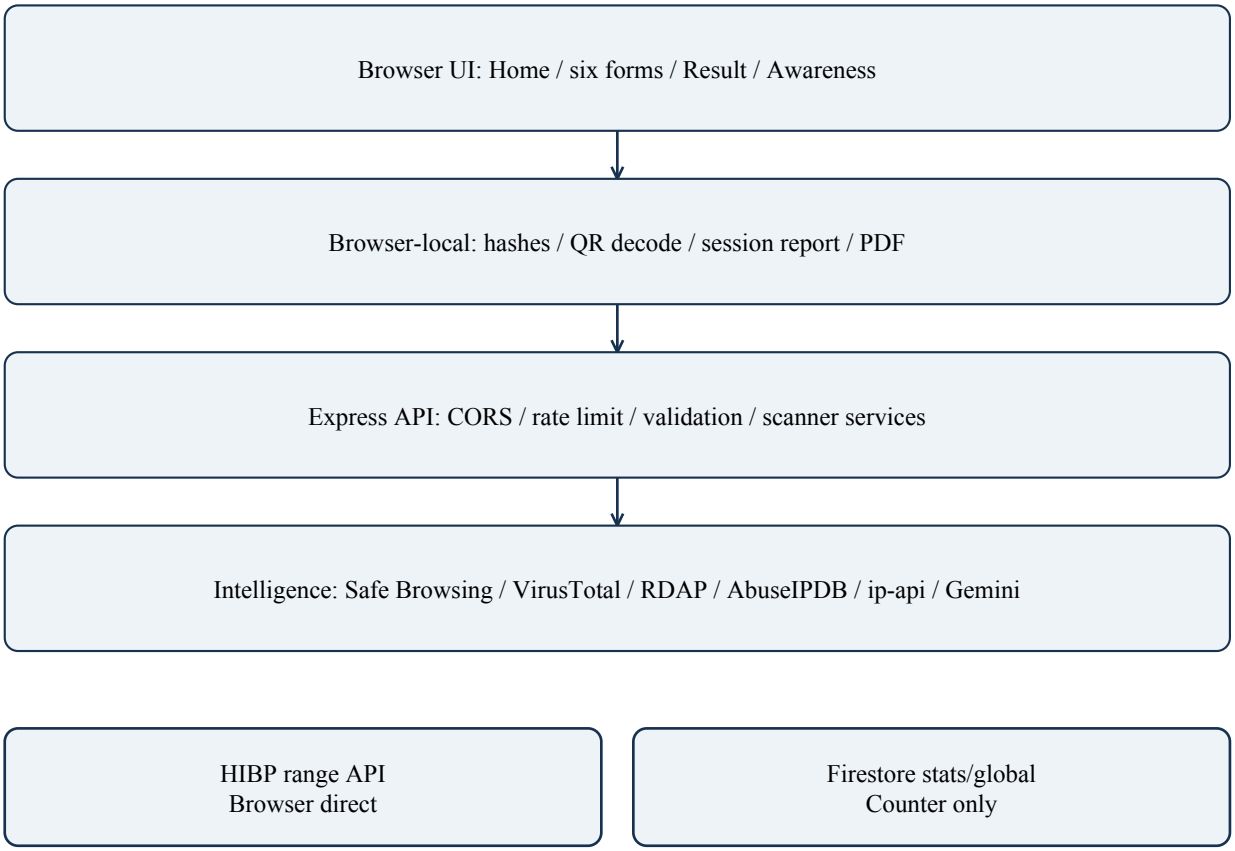


Figure 4-1: Layered architecture of CyberShield

The frontend handles input forms, local hashing and decoding, progress feedback, report storage and export. The Express backend applies CORS, size limits and per-IP request limiting before calling a scanner service. Service adapters translate provider responses into consistent availability and findings objects.

The scoring engine combines available risk signals. A failure in one source should not prevent other evidence from being presented. Firestore remains a separate, narrow concern: the browser reads and increments a public counter; it does not store raw messages, passwords or uploaded files in a scan-history collection.

4.1 Basic modules

Table 4-1: Modules and responsibilities

Module	Main responsibility
Link Scanner	URL normalisation, structural heuristics, Safe Browsing, VirusTotal and RDAP coordination.
File Scanner	Browser hash, known-file lookup, unknown-file upload/poll and metadata-preserving reports.
SMS / Email	Gemini classification, local patterns, link extraction and highlighted red flags.
QR Scanner	Image/camera decode and content dispatch to URL or local payment/content scoring.
IP Checker	Canonical IP validation, non-public exclusion, AbuseIPDB and geolocation enrichment.
Password Checker	HIBP range lookup, local comparison and strength estimate.
Score engine	Central weights, capped totals, verdict bands and breakdown lists.
Result/export	Source cards, score gauge, guidance, refresh persistence and PDF generation.
Awareness	Scam explanations and a quiz with browser-local best score.
Infrastructure	CORS, rate limit, upload limit, cache, error responses and deployments.

The service layer is divided into small modules: provider adapters perform network calls; orchestrators combine sources; score functions calculate deterministic totals; routes translate browser requests into service inputs. This separation makes provider failures and regression tests easier to reason about.

4.2 Data design and storage

The backend does not implement a relational scan-history database. It uses an in-memory TTL cache to reduce repeated provider calls. The browser keeps the most recent report in sessionStorage, and Firestore maintains a single public document for the counter.

Table 4-2: Persistent and temporary data stores

Store	Key / document	Data and purpose
Firestore	stats/global	totalScans: number. Public global completed-scan counter only.
sessionStorage	cybershield:lastResult	JSON report for the most recent scan in the current browser tab session.
localStorage	cybershield:quizBest	Best quiz score for convenience on that browser.
API memory cache	url:, file:, ip:, rdap:, geo: keys	Findings with an expiry timestamp; lost when the process restarts.
Upload memory	Multer Buffer	Temporary file bytes for the request; forwarded to VirusTotal if unknown.

4.2.1 Counter schema

Firestore counter document structure

```
stats/global
{
  "totalScans": 20
}
```

The document contains a numeric totalScans field whose value changes as scans are completed. Security rules permit reading the counter and creating it at one or updating it by exactly one. Other document paths are denied. Rules constrain the shape of the write, but the public counter is not an authenticated audit trail.

4.2.2 Report structure and integrity

Table 4-3: Shared report fields

Field	Meaning
scanType	url, file, message, qr, ip or password.
input / inputPreview	Original or normalised input and a short display value; raw password input is null.
threatScore / verdict	Bounded score and verdict, or null / unknown when no applicable score exists.
summary / explanation	Short finding summary and AI or fallback explanation.
breakdown	List of labels and awarded points.
sources	Provider/local findings, availability and relevant metadata.
unavailableSources	Human-readable list of sources that could not be checked.
scannedAt / cached	Timestamp and whether the findings were reused.
Module-specific fields	File hash/name/size, QR content/type, IP version or strength details.

4.2.3 Integrity constraints

• A supplied SHA-256 must have exactly 64 hexadecimal characters.

• Uploaded file bytes must match the supplied SHA-256 before provider submission.

• Cached detection results must retain the current request filename and MIME type.

• Equivalent IPv6 spellings are normalised before non-public classification and cache lookup.

• Passwords are cleared from the form and excluded from stored report input.

• File reports with no completed detection statistics remain unscored.

4.3 Procedural design: URL logic

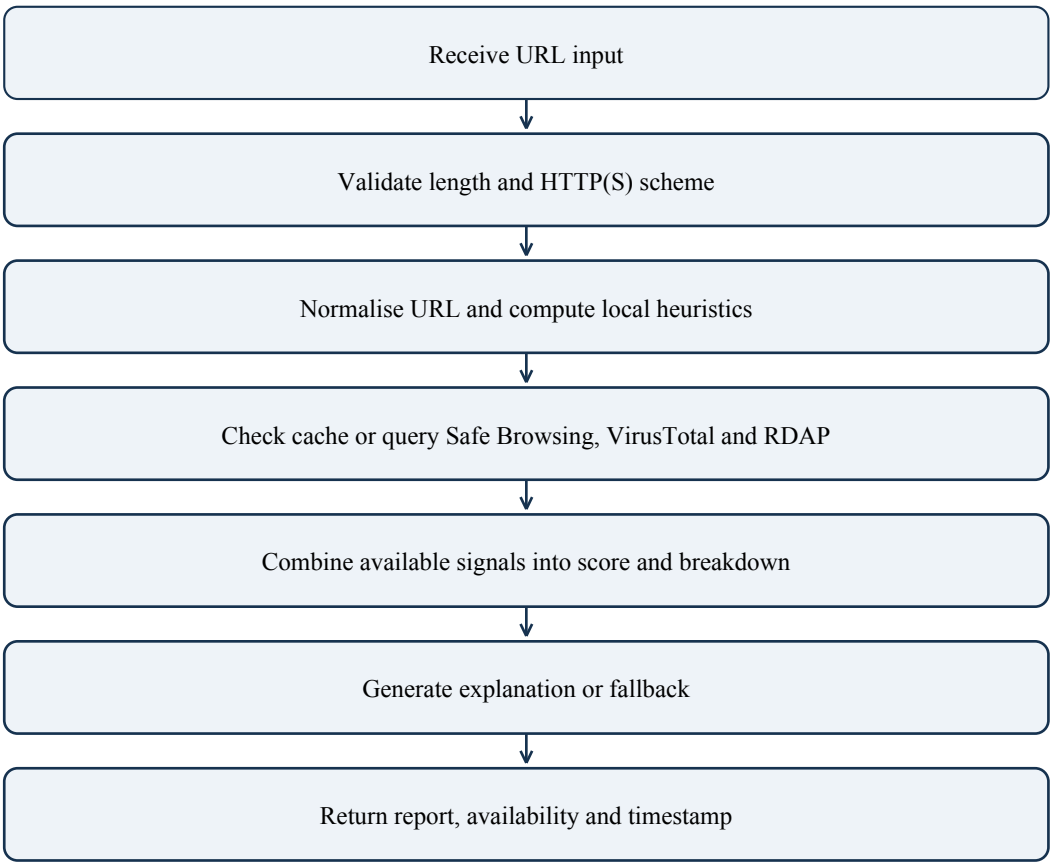


Figure 4-2: URL analysis logic flow

The URL is not downloaded directly by the Express scanner for page scraping. The service sends it to reputation APIs and queries domain registration through RDAP. The design uses reputation-service integration rather than direct website-content scraping.

A cache hit returns previously gathered findings. A fresh request can bypass the URL report cache. Queued VirusTotal results use a short cache lifetime, so the result can be checked again when engine statistics arrive. URL heuristics remain advisory: suspicious words, HTTP or Punycode alone do not prove fraud.

4.3.1 Use case model

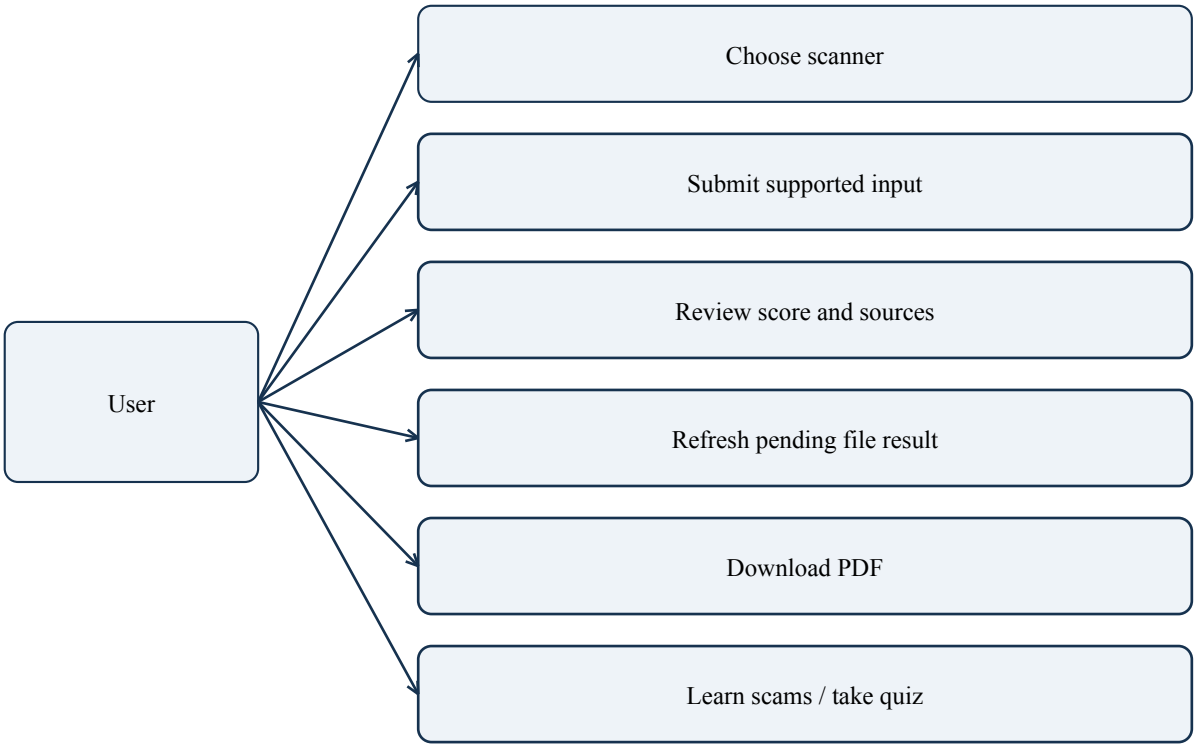


Figure 4-3: User-facing use cases

The primary actor is a user opening the website in a browser. The user chooses a scanner, supplies supported input, reviews findings, optionally refreshes a pending analysis and downloads a PDF. Awareness cards and the quiz are available independently of a scan.

External actors include security providers, the HIBP password-range service and Firestore. They supply evidence or counter data but do not control navigation. A system maintainer configures provider credentials and deployments outside the ordinary end-user interface.

Use case: scan and review

• Precondition: the browser can load the frontend; backend-backed modules also need a reachable API.

• Main flow: choose module, enter input, validate, process, render report and explain findings.

• Alternative flow: show a validation, network or rate-limit error.

• Postcondition: a completed report is visible; the latest session report can be refreshed or exported.

4.3.2 Component and module relationships

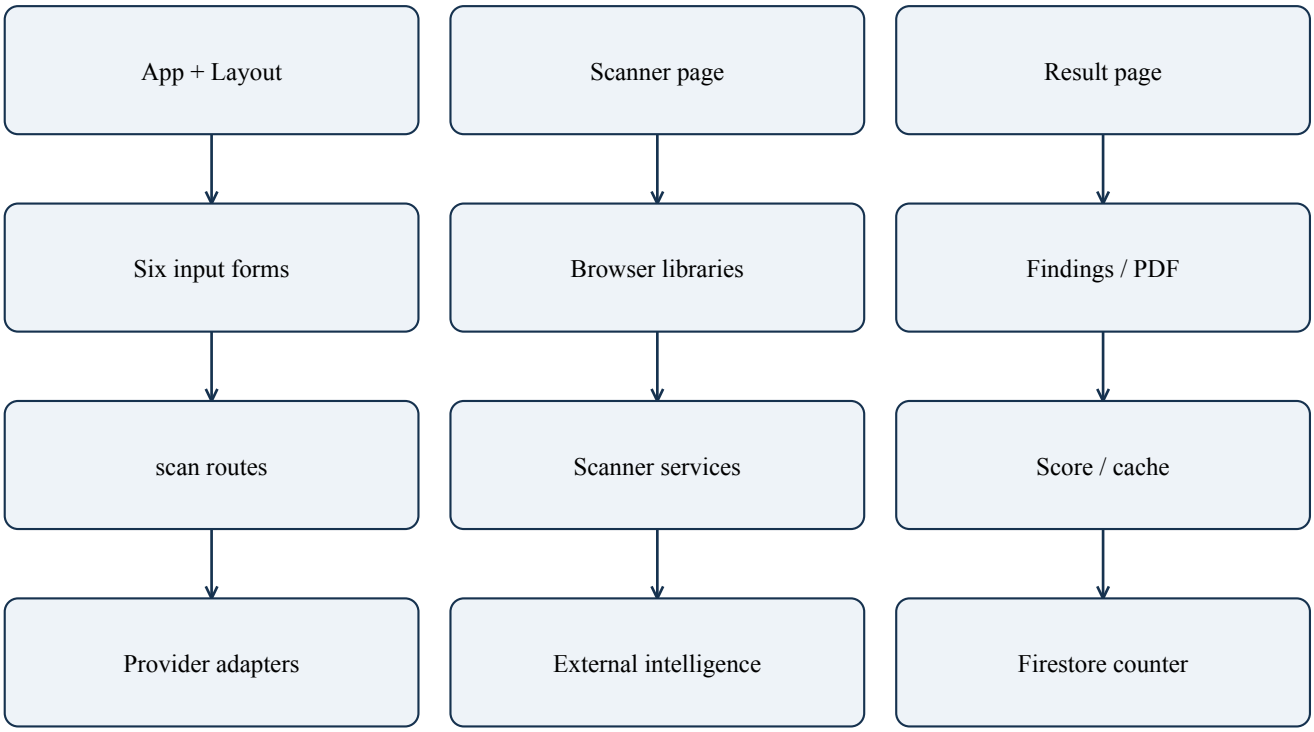


Figure 4-4: Component and service module relationships

This is a component/module diagram rather than a claim that the application implements a large inheritance-based class hierarchy. React components compose the interface; ordinary ES-module functions implement provider calls, scoring and report construction.

Scanner.jsx owns the generic runner and AbortController. Each form passes a validated input to a module-specific handler. Result.jsx selects the appropriate findings component according to scanType. Backend scan routes dispatch to orchestrators, which use service adapters and the shared scoring engine.

The modules communicate through explicit function arguments and JSON objects. Provider response details are normalised before being displayed, which prevents the UI from depending directly on every provider's original response format.

4.3.3 Sequence of a URL scan

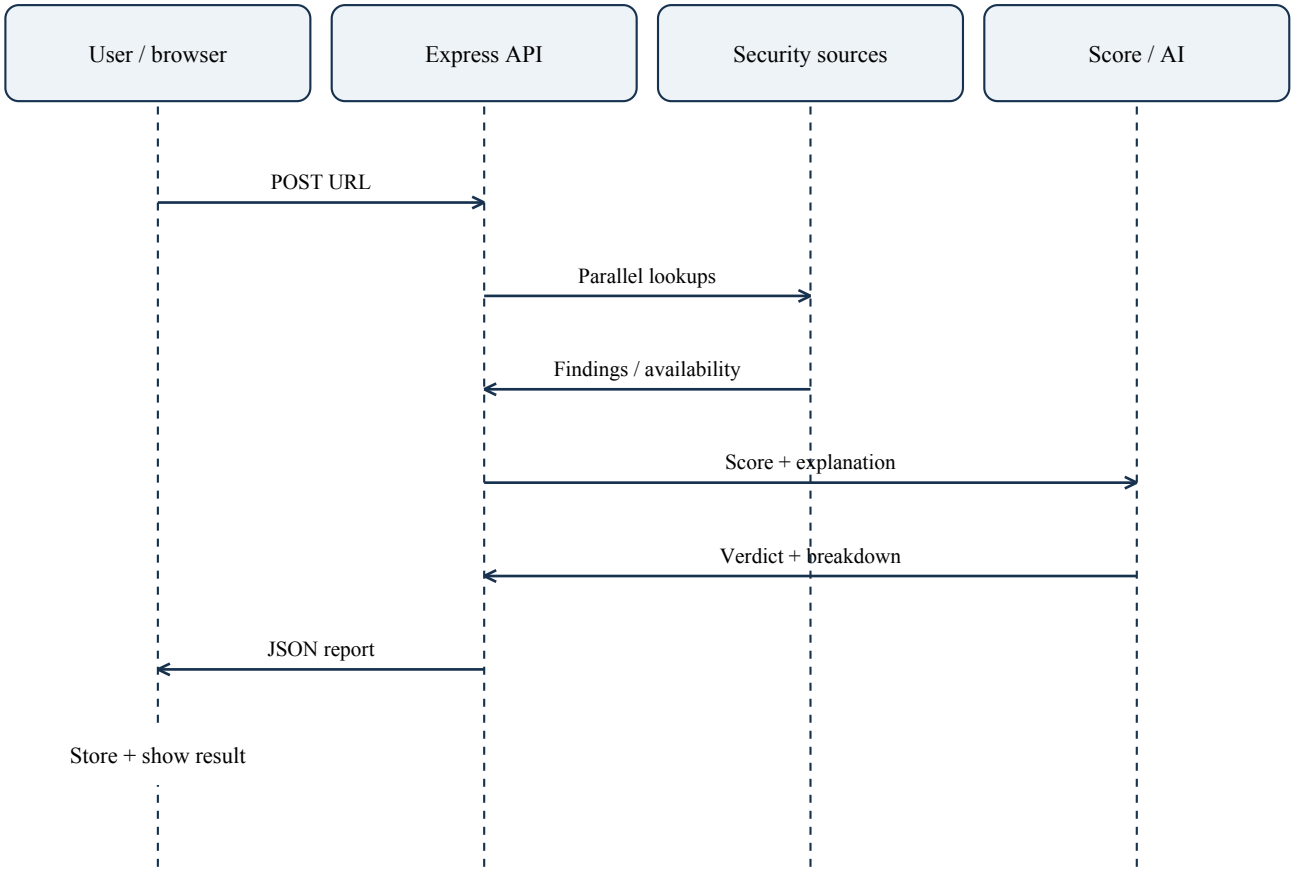


Figure 4-5: URL scan sequence across system boundaries

The browser sends one JSON scan request to the API. The middleware validates the origin and request limits. The URL orchestrator normalises the input, checks cache and starts independent intelligence requests. It then passes available findings to the score engine and requests an explanation.

The API returns the report. The frontend observes a minimum radar delay, checks whether the scan was cancelled, stores the completed report and navigates to Result. A cancelled scan must not save data, increment the counter or navigate after the user has left.

Provider calls are bounded by individual timeouts. A late or unavailable AI explanation can be replaced with a local template. A queued VirusTotal analysis is communicated as pending, with a way to retrieve updated results.

4.3.4 Data flow diagrams

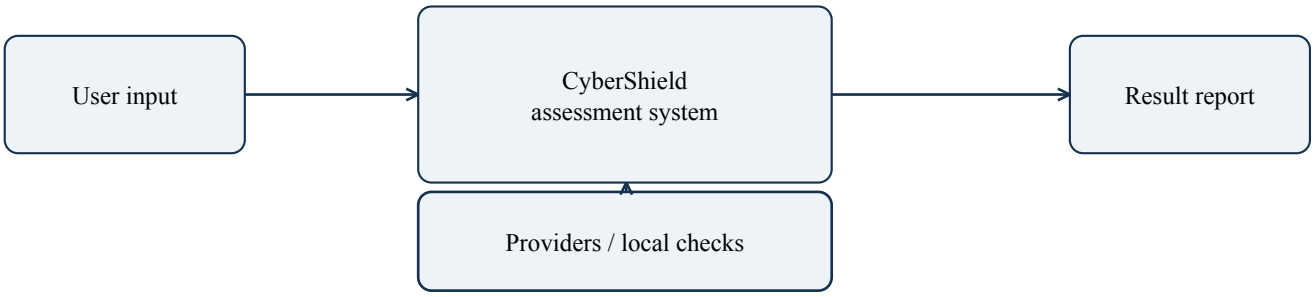


Figure 4-6: Context data flow diagram (Level 0)

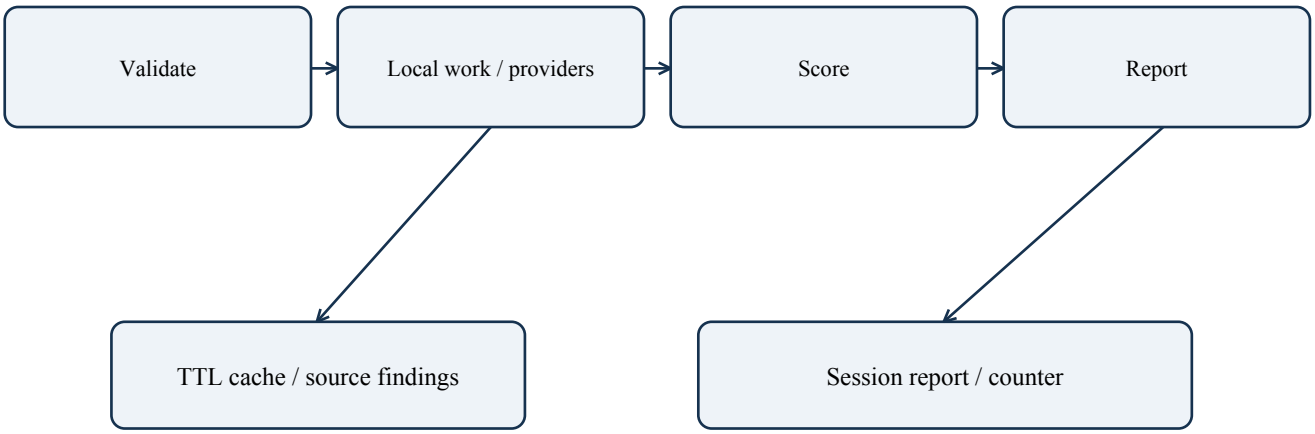


Figure 4-7: Scanner data flow diagram (Level 1)

The context diagram shows input flowing from the user to CyberShield and a report flowing back. The detailed diagram separates validation, local processing, provider orchestration, scoring and presentation. Firestore counter updates and browser-session persistence are side effects of completed scans.

Only data required by a specific module crosses its boundary. A raw password is not included in a backend request. A QR containing a web link sends the destination to the URL API; other QR content can remain local.

4.3.5 File-analysis activity

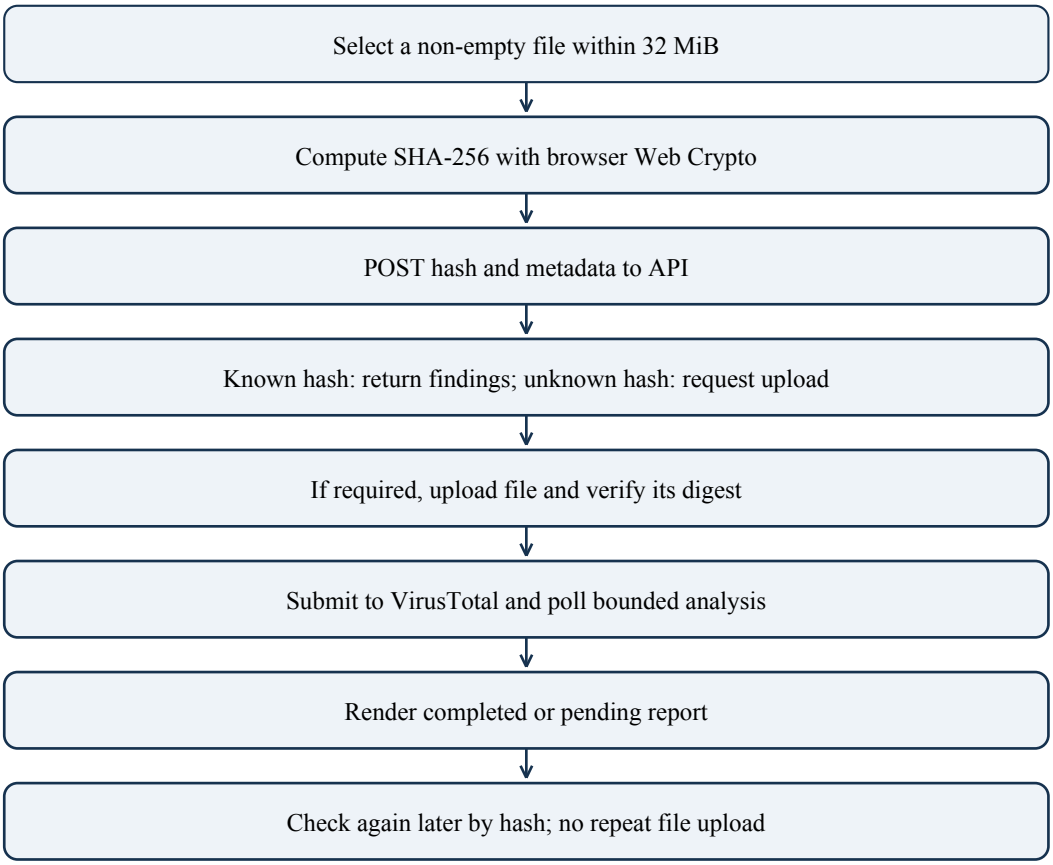


Figure 4-8: Hash-first file analysis activity

The hash-first approach avoids uploading known content unnecessarily. Uploads are held in memory by the API rather than written to a project folder. An unknown file is nevertheless transferred to VirusTotal, so the interface must make that external analysis transfer clear.

A pending report does not imply a clean file. The Check again action repeats a JSON hash lookup, and a completed result replaces the earlier unscored report.

4.3.6 Data relationships

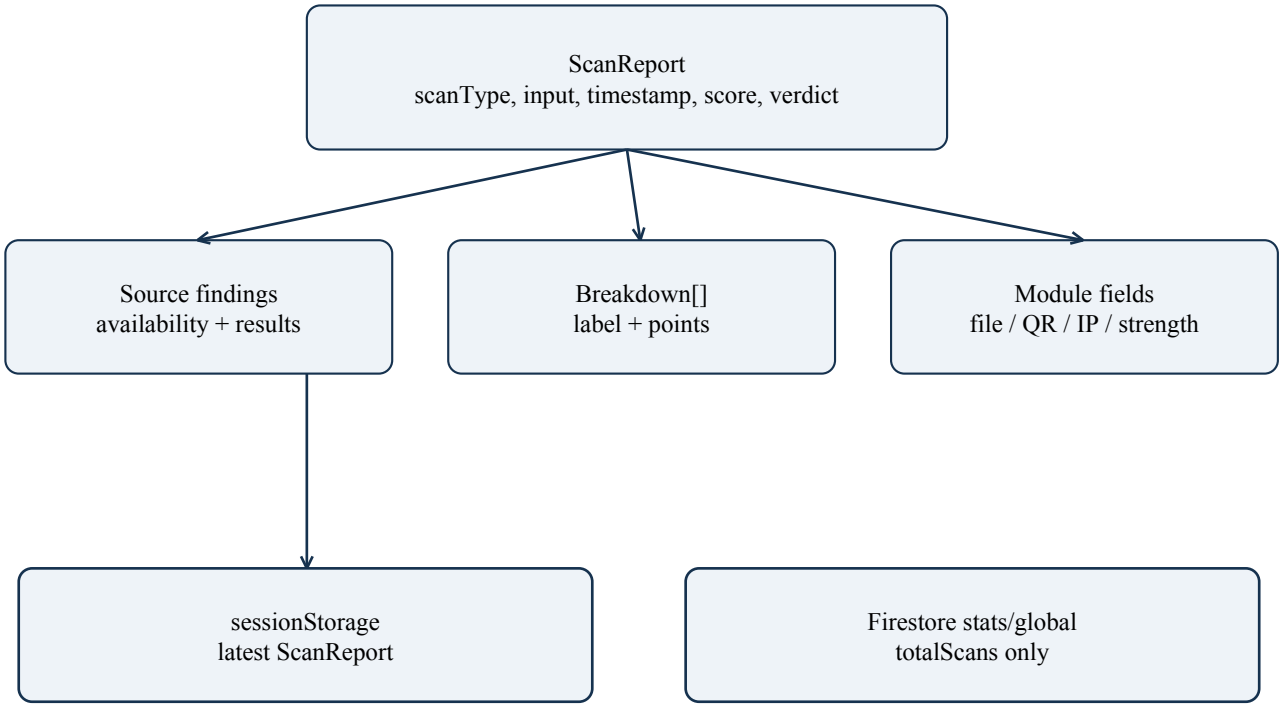


Figure 4-9: Conceptual report and storage relationships

The report is an aggregate of a scan identity, score, explanation, breakdown and source findings. It may contain module-specific fields such as a file fingerprint or parsed UPI information. There is no SQL scan_history table or relational foreign-key enforcement in this implementation.

The latest report is serialised into browser session storage. Source findings and score-breakdown entries are embedded arrays/objects. The public Firestore stats document is independent of any user identity or report content and contains only totalScans.

Table 4-4: Important data structures

Structure	Purpose
URL object	Parsed scheme, host, path, query and userinfo.
Provider result object	Availability, status and source-specific findings.
breakdown[]	Ordered risk contributions with label and points.
Map with expires	In-memory TTL cache.
AbortController	Cancellation signal for the active scan.
QR parse object	Classified content and payment/network fields.

4.3.7 URL and IP algorithms

Algorithm 1: URL normalisation and scoring

```
INPUT: raw URL string
1. Trim input and reject empty / overlong values.
2. If no scheme is supplied, prefix https://.
3. Parse a URL and accept only http: or https:.
4. Validate the host, including an IPv6 literal.
5. Reuse cached findings unless a fresh scan is asked.
6. Query relevant providers and compute heuristics.
7. Sum weighted signals and clamp the score to 0..100.
8. Map score to a verdict; build the breakdown.
9. Attach explanation and source availability.
OUTPUT: consistent report object
```

Algorithm 2: Public IP reputation

```
INPUT: IPv4 or IPv6 text
1. Trim the input and remove optional IPv6 brackets.
2. Canonicalise IPv6 and convert IPv4-mapped values.
3. Reject an invalid address.
4. If non-public, return score=null and unknown verdict.
5. Otherwise query AbuseIPDB and geolocation.
6. If reputation is unavailable, return an unscored report.
7. Score abuse confidence plus report-volume bonus.
OUTPUT: reputation report with location enrichment
```

Canonicalisation is important because textual spelling is not the address identity. Expanded ::1, leading zeroes in IPv6 groups and hexadecimal IPv4-mapped forms must receive the same classification as their usual compact representation.

4.3.8 File, message and password algorithms

Algorithm 3: Hash-first file handling

```
1. Validate file size and compute the browser SHA-256.
2. Send hash and metadata for lookup.
3. If known, return source findings without file bytes.
4. If unknown, upload bytes for fresh provider analysis.
5. Verify bytes match the requested hash on the server.
6. Poll a bounded number of times.
7. Show completed statistics or an unscored pending state.
```

Algorithm 4: Message classification

```
1. Trim message and enforce the 5,000-character cap.
2. Extract up to three links.
3. Run Gemini analysis and embedded-link scans.
4. Apply deterministic scam patterns.
5. Merge red flags; retain stronger local risk evidence.
6. Add half the highest embedded-link risk score.
7. Clamp the total and construct the result report.
```

Algorithm 5: Password range lookup

```
1. Compute SHA-1 locally and split prefix / suffix.
2. Request only the first five hash characters from HIBP.
3. Compare returned suffixes inside the browser.
4. Estimate local strength without a provider request.
5. Score breach exposure or local weakness.
6. Store a masked preview, never the raw password.
```

These algorithms define the implemented processing workflow. Detection coverage depends on the provider data and the configured scoring rules.

4.3.9 Threat-score weights

Table 4-5: URL and backend scoring weights

Signal	Points / rule
Google Safe Browsing match	+50
VirusTotal URL flags	+10 per malicious/suspicious engine; maximum +40
Domain age under 30 days	+15
HTTP rather than HTTPS	+5
Suspicious URL keywords	+10
Raw IP URL host	+10
Three or more subdomains	+5
URL userinfo or Punycode host	+10
File detections	8 points per detecting engine, capped at 100
IP reputation	Abuse confidence score; +10 if reports exceed 20
Message	AI/local assessment plus half the worst embedded-link score

Verdict thresholds

Table 4-6: Verdict bands

Score / state	Verdict	Interpretation
0 - 30	Safe	Lower observed risk; not a guarantee.
31 - 60	Suspicious	Warnings warrant independent verification.
61 - 100	Dangerous	Strong warning signals; avoid acting on the input.
No applicable score	Not scored	Reputation was unavailable, pending or not applicable.

Weights are deterministic and centralised in threatScore.js and localScore.js. An authoritative source finding remains visible in its own card even when the numeric thresholds produce a different overall severity. Calibrating these weights on a representative dataset is a future task.

4.3.10 Local QR/password scoring

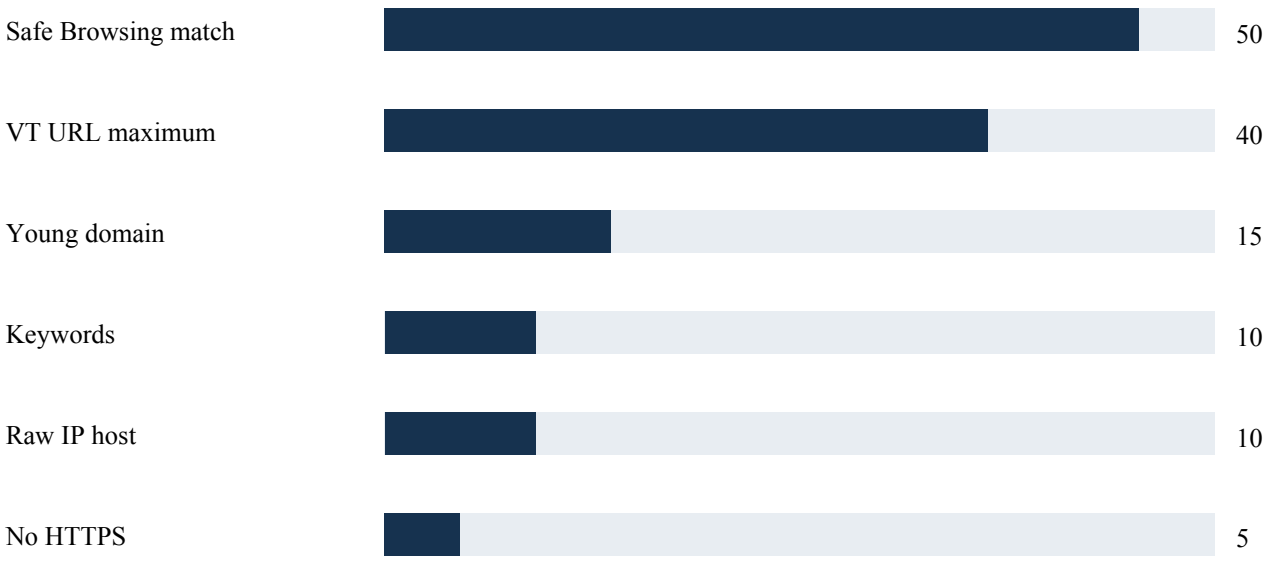


Figure 4-10: Representative URL risk weights

Table 4-7: Local scoring rules

Module	Signal	Points
UPI QR	Payment request	30
UPI QR	Positive pre-filled amount	15
UPI QR	Unknown/missing handle	10
UPI QR	Scam words in name or note	15
UPI QR	Personal-looking name with fixed amount	10
Password	Known breach base	70
Password	Breach-count magnitude bonus	Up to +30
Password	Very weak / weak / fair / strong / very strong	45 / 35 / 20 / 5 / 0
Other QR	Open Wi-Fi / plain text	20 / 5

UPI rules identify contextual warning signs, not the legitimacy of a merchant. A payment request may be ordinary when the user intends to pay. Password strength is an approximation, and a missing breach match does not prove a password has never been exposed.

4.4 Interface design and AI role

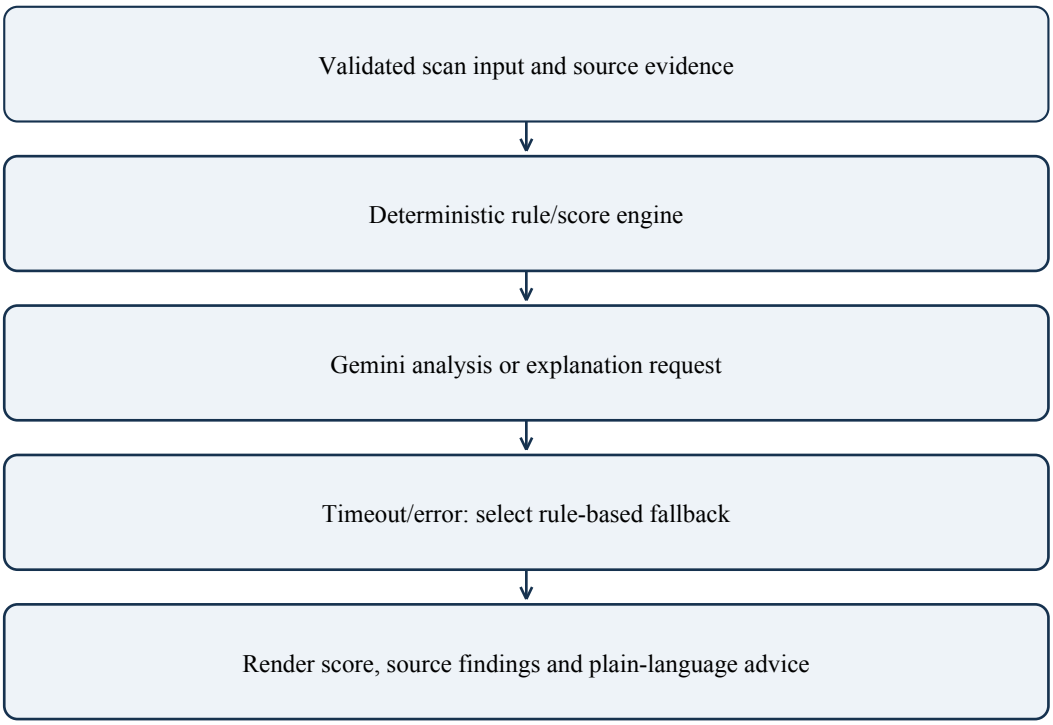


Figure 4-11: Provider-assisted explanation pipeline

The interface uses a common scanner page with six route-driven tabs. Input labels, assistance controls and validation messages are placed close to the field. The scanning overlay communicates progress, and Result groups the main verdict, explanation, breakdown and source-specific cards.

Gemini is integrated as an external generative model for message analysis and short explanations. Message requests use structured JSON, while explanation requests produce plain-language text. The application combines model output with deterministic evidence, and stronger local scam patterns can raise a low AI assessment.

Page content is not hidden while an old route exits. A visible loading fallback remains inside the main area during lazy downloads. This design prevents the blank-page state observed during normal navigation.

4.5 Security issues and test-case design

Table 4-8: Security controls and scope

Concern	Implemented control / limitation
Provider credentials	Private keys stay in server environment variables; browser Firebase config is public.
File identity	Validate SHA-256 and verify uploaded bytes; preserve current metadata in cache hits.
Input abuse	HTTP(S) scheme validation, character caps, file-size limit and small JSON body limit.
Quota exhaustion	20 backend requests per 10 minutes per IP and report caching; no full shared-provider scheduler.
Network origin	CORS allows the configured GitHub Pages origin; server-to-server requests may have no Origin.
Password privacy	Only a five-character SHA-1 prefix leaves the browser.
Messages and files	Message text may reach Gemini; unknown file bytes may reach VirusTotal.
Non-public IPs	Canonical IPv6 classification; no public-reputation score for local/reserved addresses.
Cancelled work	Abort signal checked before saving a report, incrementing the counter or navigating.
PDF encoding	Use INR text for unsupported rupee glyph; font coverage is still limited.

Test cases cover valid inputs, malformed data, unavailable sources, cache reuse, pending analysis, cancellation and navigation. Privacy checks use designated test passwords and non-sensitive text files. Camera decoding is verified with a controlled QR-video input; physical camera performance remains a separate device check.

CHAPTER 5
IMPLEMENTATION & TESTING

5.1 Implementation environment

The implementation is organised as a client/server repository. The frontend is built into static assets; the API remains a separate Node process. npm scripts at the root coordinate installation, development, build, lint and tests.

Table 5-1: Verified implementation environment

Area	Technology
Frontend	React 19, React Router 7, Vite 8 and Tailwind CSS 4
Animation	Framer Motion 14
API	Node.js, Express 5 and Axios
Security adapters	Safe Browsing v4, VirusTotal v3, AbuseIPDB v2, RDAP and ip-api
AI	Google GenAI SDK with configured Gemini models and fallback logic
Browser processing	Web Crypto, Canvas, jsQR and getUserMedia
Reports	jsPDF 4 and jsPDF-AutoTable 5
Counter	Firebase SDK / Firestore stats/global
Checks	Node test runner, oxlint and Chrome browser verification
Hosting	GitHub Pages and Render Free Node Web Service

The tested source uses ES modules. Lockfiles support reproducible dependency installation. Node 22 is configured on Render and in the GitHub build workflow, avoiding the outdated Node 18 requirement that appeared in the initial documentation.

5.1.1 Project organisation

Table 5-2: Project files and their purpose

File / folder	Purpose
client/src/App.jsx	Route map and lazy page imports.
client/src/pages/Scanner.jsx	Input-module selection, runner, cancellation and navigation.
client/src/pages/Result.jsx	Shared results, module findings and pending-analysis refresh.
client/src/components/scanner/	Six forms, camera decoding, radar overlay and errors.
client/src/lib/	Hashing, password/QR handling, local scores, report storage and PDF.
client/src/services/	API fetch wrapper and public-counter increment.
server/index.js	CORS, JSON limit, route mount, health check and error handling.
server/routes/scan.js	URL, file, message and IP request dispatch.
server/services/	Provider adapters and scanner orchestrators.
server/utils/	Threat scoring, URL/message rules, cache and configuration.
server/middleware/	Rate limit, upload parsing and safe error responses.
client/test/ and server/test/	Focused automated regression tests.
render.yaml / pages.yml	Backend blueprint and frontend deployment workflow.
firestore.rules	Public-counter data constraints; deny other paths.

The separation keeps UI changes independent of provider adapters and keeps score weights inspectable. The same report contract supports browser-only modules and API-backed modules.

5.2 Code implementation: API limits

The following source-code excerpts document the implemented responsibilities. Private environment values are excluded from the report.

Per-IP scanner rate limiter (server/middleware/rateLimiter.js)

```
const windowMinutes = Number(process.env.RATE_LIMIT_WINDOW_MINUTES) || 10;
const max = Number(process.env.RATE_LIMIT_MAX) || 20;

export const scanLimiter = rateLimit({
  windowMs: windowMinutes * 60 * 1000,
  limit: max,
  standardHeaders: 'draft-7',
  legacyHeaders: false,
  message: {
    error: 'rate_limited',
    message: `Too many scans. You can run ${max} scans every ${windowMinutes}
   minutes — please wait a bit.`,
  },
});
```

Memory-only file upload (server/middleware/upload.js)

```
export const MAX_FILE_BYTES = 32 * 1024 * 1024;

export const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: MAX_FILE_BYTES, files: 1 },
});

/** Translate multer errors into our JSON error shape. */
```

Implementation notes

The limiter applies before scanner orchestration, so cached requests also count. A hash-first unknown file uses two backend requests: lookup and upload. Rate limiting is per calling IP, while provider quotas are shared across the configured account; these are different controls.

The upload limit is 32 MiB and the JSON body limit is 64 KiB. Files are held in a Buffer for the duration of the request. Error handling translates upload failures into a user-readable JSON response and avoids sending a stack trace to the frontend.

5.2.1 URL validation

Backend URL normalisation (server/utils/urlUtils.js)

```
export function normalizeUrl(raw) {
  if (typeof raw !== 'string') return null;
  let input = raw.trim();
  if (!input || input.length > 2048) return null;
  // No scheme typed (e.g. "example.com") \u2192 assume https so the user is
  not penalised for our guess
  if (!/^[a-z][a-z0-9+.-]*:\/\//i.test(input)) input = `https://${input}`;
  try {
    const u = new URL(input);
    if (!['http:', 'https:'].includes(u.protocol)) return null;
    if (!u.hostname || !u.hostname.includes('.') && !isIpAddress(u.hostname))
   return null;
    return u;
  } catch {
    return null;
  }
}
```

Frontend validation (client/src/lib/urlValidation.js)

```
import { isValidIp } from './ipValidation.js';

/** Match the API's supported HTTP(S) URLs, including bare domains and IPv6
  hosts. */
export function isValidHttpUrl(raw) {
  const value = raw.trim();
  if (!value || value.length > 2048) return false;
  const input = /^[a-z][a-z0-9+.-]*:\/\//i.test(value) ? value :
  `https://${value}`;
  try {
    const u = new URL(input);
    return ['http:', 'https:'].includes(u.protocol)
      && (u.hostname.includes('.') ||
  isValidIp(u.hostname.replace(/^\[|\]$/g, '')));
  } catch {
    return false;
  }
}
```

Both ends accept HTTP(S) schemes, normal domains and valid IP hosts. A bare domain is treated as HTTPS. The UI rejects malformed input before a network request, while the backend validates again because a caller can bypass the form.

The IPv6 regression verifies that frontend and backend acceptance rules agree. A hostname test that requires a dot would incorrectly reject an IPv6 literal such as [2606:4700:4700::1111].

5.2.2 Score computation

Score and verdict boundaries (server/utils/threatScore.js)

```
  SAFE_MAX: 30,        // 0–30  \u2192 Safe
  SUSPICIOUS_MAX: 60,  // 31–60 \u2192 Suspicious, 61–100 \u2192 Dangerous
};

export function verdictFromScore(score) {
  if (score <= THRESHOLDS.SAFE_MAX) return 'safe';
  if (score <= THRESHOLDS.SUSPICIOUS_MAX) return 'suspicious';
  return 'dangerous';
}

const clamp = (n) => Math.max(0, Math.min(100, Math.round(n)));

/**
```

File scoring guard and detection points

```
      score = Math.min(W.VT_MAX, flagged * W.VT_PER_ENGINE);
      breakdown.push({ label: `${flagged} antivirus engine${flagged > 1 ? 's'
   : ''} detected malware`, points: score });
    }
  }
  const finalScore = clamp(score);
  return { score: finalScore, verdict: verdictFromScore(finalScore),
  breakdown };
}

/** IP score: AbuseIPDB confidence + report-volume bonus. */
export function scoreIp({ abuseIpdb }) {
  const W = WEIGHTS.ip;
  const breakdown = [];
  let score = 0;
  if (abuseIpdb?.available) {
    score = (abuseIpdb.abuseConfidenceScore || 0) *
  W.ABUSE_CONFIDENCE_MULTIPLIER;
    if (score > 0) breakdown.push({ label: `AbuseIPDB confidence
  ${abuseIpdb.abuseConfidenceScore}%`, points: Math.round(score) });
    if ((abuseIpdb.totalReports || 0) > W.MANY_REPORTS) {
      score += W.MANY_REPORTS_BONUS;
      breakdown.push({ label: `${abuseIpdb.totalReports} abuse reports in 90
  days`, points: W.MANY_REPORTS_BONUS });
    }
```

The scoring functions return score, verdict and breakdown rather than only a label. A bounded integer score is easier to display, compare and explain. The score remains a hand-weighted indicator, not an empirically calibrated probability.

A file with no usable engine statistics receives a null score and unknown verdict. This prevents queued or unavailable detection data from being displayed as a completed clean scan. The same principle applies to unavailable or inapplicable public-IP reputation.

5.2.3 URL orchestration and source availability

Parallel URL-source requests (server/services/urlScanner.js)

```
  const domain = u.hostname;
  const registrable = getRegistrableDomain(domain);
  const heuristics = analyzeUrlHeuristics(u);

  // All network sources in parallel
  const [safeBrowsing, virusTotal, domainAge] = await Promise.all([
    settled(checkSafeBrowsing(url)),
    settled(scanUrlWithVirusTotal(url, { quick })),
    heuristics.isIpUrl ? Promise.resolve({ available: false, reason: 'ip_url'
   }) : settled(getDomainAge(registrable)),
  ]);

  const { score, verdict, breakdown } = scoreUrl({ safeBrowsing, virusTotal,
  domainAge, heuristics });

  // quick mode (links inside a message): skip the AI explanation — the
  message itself gets one
  const ai = quick
    ? { available: false, text: fallbackExplanation({ scanType: 'url',
  verdict, score, breakdown }) }
    : await explainFindings({
    scanType: 'url',
    input: url,
```

VirusTotal completed-statistics guard

```
    });
}

export function fromAttributes(attrs, extra = {}) {
  const rawStats = attrs.last_analysis_stats || attrs.stats;
  const hasStats = rawStats && (!attrs.status || attrs.status ===
  'completed')
    && ['malicious', 'suspicious', 'harmless', 'undetected'].some((category)
  => rawStats[category] > 0);
  const stats = hasStats ? rawStats : {};
  const results = attrs.last_analysis_results || attrs.results || {};
  return {
    available: true,
    status: hasStats ? 'completed' : 'queued',
    stats: hasStats ? {
      malicious: stats.malicious || 0,
      suspicious: stats.suspicious || 0,
      harmless: stats.harmless || 0,
      undetected: stats.undetected || 0,
      timeout: stats.timeout || 0,
    } : null,
    engines: toEngineList(results),
    ...extra,
```

The URL scanner normalises provider failures into unavailable result objects. Independent checks run in parallel. It combines the findings, builds source cards and uses an AI explanation or deterministic fallback text.

VirusTotal statistics must contain an actual classification and correspond to a completed analysis. Zero-only, unfinished or failed-only statistics are not sufficient to produce a malware verdict. The browser can request an updated report after analysis completes.

5.2.4 File identity and hash-first handling

Digest verification and current file metadata

```
if (!SHA256_RE.test(sha256 || '')) throw httpError(400, 'invalid_hash', 'A
valid SHA-256 hash is required.');
const hash = sha256.toLowerCase();
if (buffer && createHash('sha256').update(buffer).digest('hex') !== hash) {
  throw httpError(400, 'hash_mismatch', 'The uploaded file does not match
its SHA-256 hash.');
}
const safeName = String(fileName || 'file').slice(0, 120);

const cacheKey = `file:${hash}`;
const cached = cacheGet(cacheKey);
if (cached) return {
  ...cached,
  input: safeName,
  inputPreview: safeName,
  fileName: safeName,
  size: size ?? cached.size,
  sizeLabel: prettyBytes(size ?? cached.size),
  mimeType: mimeType || cached.mimeType,
  cached: true,
};

// Step 2: hash lookup
let vt = await lookupFileHash(hash);

// Step 3: unknown + we have the bytes \u2192 upload. Unknown + no bytes
\u2192 ask the client to send them.
if (vt.available && vt.status === 'not_found') {
```

Unknown-file branch

```
  buffer = null; // explicit: let the Buffer be garbage-collected
immediately
}

const { score, verdict, breakdown } = scoreFile({ virusTotal: vt });
const flagged = vt.stats ? vt.stats.malicious + vt.stats.suspicious : 0;

const ai = score == null ? {
  available: false,
  text: vt.status === 'queued'
```

The browser calculates the hash before any file upload. The backend recomputes the digest when bytes are supplied and rejects a mismatch. This keeps provider submission and cached findings aligned with the actual content.

Detection evidence is reusable by hash, but the name and MIME type are properties of the current request. A second scan of the same bytes under a different filename must therefore receive its current filename, while retaining the cached detection results.

5.2.5 Message rules and AI

Message input and parallel analysis

```
const MAX_CHARS = 5000;
const MAX_LINKS = 3; // keep VirusTotal quota in check

export async function scanMessage(rawText) {
  const text = String(rawText || '').trim();
  if (!text) throw httpError(400, 'missing_text', 'Paste the SMS or email
  text to check.');
  if (text.length > MAX_CHARS) throw httpError(400, 'text_too_long', `Message
   is too long (max ${MAX_CHARS} characters).`);

  const urls = extractUrls(text).slice(0, MAX_LINKS);

  // AI + rules + link scans all in parallel
  const [aiRaw, linkReports] = await Promise.all([
    analyzeMessage(text),
    Promise.all(urls.map((u) => scanUrl(u, { quick: true }).catch(() =>
  null))),
  ]);
  const rules = analyzeMessageHeuristics(text);

  // Prefer Gemini; fall back to rules. Merge rule red-flags the AI missed.
  const ai = aiRaw.available ? { ...aiRaw, redFlags: [...aiRaw.redFlags],
  source: 'gemini' } : rules;
  if (aiRaw.available) {
    const seen = new Set(ai.redFlags.map((f) => f.phrase.toLowerCase()));
    for (const f of rules.redFlags) if (!seen.has(f.phrase.toLowerCase()))
  ai.redFlags.push(f);
  }
```

Message score combines strongest assessment and link evidence

Local patterns cover KYC, OTP requests, fake jobs, urgency, courier fees, loan offers and UPI refund traps. Protective advice such as "do not share your OTP" should not be interpreted as asking for an OTP. A dedicated regression checks this distinction.

Gemini returns structured message findings when available. Local red flags are merged into the response, and stronger local risk evidence can raise a low AI assessment. The report identifies whether Gemini or rule-based analysis was used and displays exact matched phrases.

5.2.6 Password privacy

Password hash-range lookup (client/src/lib/password.js)

```
export async function checkPwned(password, signal) {
  const hash = await sha1HexUpper(password);
  const prefix = hash.slice(0, 5);
  const suffix = hash.slice(5);
  let res;
  try {
    res = await fetch(HIBP_RANGE + prefix, { headers: { 'Add-Padding': 'true'
   }, signal });
  } catch (err) {
    if (err.name === 'AbortError') throw err;
    return { available: false, reason: 'network', prefix, breachCount: 0,
  candidates: 0 };
  }
  if (!res.ok) return { available: false, reason: `http_${res.status}`,
  prefix, breachCount: 0, candidates: 0 };
  const body = await res.text();
  let breachCount = 0;
  let candidates = 0;
  for (const line of body.split('\n')) {
    const [sfx, count] = line.trim().split(':');
    if (!sfx) continue;
    const n = Number(count) || 0;
    if (n > 0) candidates++; // padded entries have count 0
    if (sfx === suffix) breachCount = n;
  }
  return { available: true, prefix, breachCount, candidates };
}
```

The SHA-1 digest is calculated with Web Crypto. Only the first five uppercase hexadecimal characters are sent to HIBP. The suffix comparison occurs locally after the API response; padded zero-count records are ignored when counting useful candidates.

The Password form clears its value immediately after submission. The saved report has input set to null and a masked preview. The local strength estimator checks character classes, common words, repetition, sequences and simple patterns. The crack-time value is a heuristic estimate based on the implemented assumptions.

If HIBP cannot be reached, exposure remains unknown and the report explains that limitation. A local strength score may still be available; that should not be confused with a successful breach-database lookup.

5.2.7 QR dispatch and scan cancellation

QR payment schemes and parsing guard (client/src/lib/qr.js)

```
const UPI_RE = /^(upi|gpay|paytmmp|phonepe|bhim|tez):\/\//i;

export function parseQrContent(raw) {
  const text = String(raw || '').trim();
  if (!text) return { kind: 'text', raw: text };

  if (UPI_RE.test(text)) {
    const query = text.includes('?') ? text.slice(text.indexOf('?') + 1) :
  '';
    const p = new URLSearchParams(query);
    const amount = p.get('am') ? Number(p.get('am')) : null;
    return {
      kind: 'upi',
```

Cancellation checked before saving a result

```
    setScanning(true);
    abortRef.current?.abort();
    const controller = new AbortController();
    abortRef.current = controller;
    try {
      const [result] = await Promise.all([fn(controller.signal),
  wait(MIN_SCAN_MS)]);
      // Local and cached scans can finish before the radar delay. Leaving
  the
      // page during that delay must still cancel saving and navigation.
      controller.signal.throwIfAborted();
      saveLastResult(result);
      bumpGlobalCounter(); // fire-and-forget public counter
      navigate('/result', { state: { result } });
    } catch (err) {
      if (!controller.signal.aborted && err.name !== 'AbortError') setError({
   type, value: err });
    } finally {
      if (!controller.signal.aborted) setScanning(false);
    }
  };

  const handleUrlScan = (url) => runScan((signal) => api.scanUrl(url, {
  signal }), { preview: url });
```

Only recognised payment schemes enter the UPI parser. HTTP(S) links with a pa query parameter remain web links and are sent through the Link Scanner. QR images and camera frames are decoded locally using Canvas and jsQR.

Cancellation must also cover fast local scans. A result can resolve before the cosmetic minimum radar delay finishes. Checking the AbortController after that delay prevents a late session write and navigation when the user has already left the scanner.

5.2.8 IP, PDF and route corrections

IPv6 canonicalisation (server/services/ipScanner.js)

```
function normalizeIp(ip) {
  if (isIP(ip) !== 6) return ip;
  // URL parsing compresses zero groups and removes leading zeroes. It also
  // converts dotted IPv4-mapped addresses to the equivalent hexadecimal
  form.
  const host = new URL(`http://[${ip.split('%')[0]}]/`).hostname.slice(1,
  -1);
  const mapped = host.match(/^::ffff:([a-f0-9]+):([a-f0-9]+)$/i);
  if (!mapped) return host;
  const high = parseInt(mapped[1], 16);
  const low = parseInt(mapped[2], 16);
  return [high >> 8, high & 255, low >> 8, low & 255].join('.');
}

export function isNonPublicIp(ip, version = isIP(ip)) {
  if (version === 6) {
    ip = normalizeIp(ip);
```

Readable PDF currency (client/src/lib/pdfReport.js)

```
const fmtDate = (iso) => new Date(iso).toLocaleString('en-IN', { dateStyle:
  'medium', timeStyle: 'short' });

// The built-in PDF fonts cannot encode the rupee or arrow glyphs. Keep the
// currency explicit and readable in both document text and table cells.
export const pdfText = (value) => String(value ?? '').replaceAll('\u20b9',
  'INR ').replaceAll('\u2192', '->');
```

Visible lazy-route container (Layout.jsx)

```
      <main id="main" tabIndex={-1} className="flex-1 px-4 sm:px-6">
        <div className="mx-auto w-full max-w-7xl">
          <ErrorBoundary key={location.pathname}>
            <Suspense fallback={<PageLoader />}>
              <Outlet />
            </Suspense>
          </ErrorBoundary>
        </div>
      </main>
```

The normalised IP is classified before external lookup and cache access. PDF text substitutes INR for the unsupported rupee glyph in the built-in font. The route shell no longer waits for an exiting animation; a local Suspense boundary shows a visible loader while the page chunk downloads.

These changes address observed defects rather than hypothetical improvements. The next section records how the corrected behaviour was checked.

5.3 System testing approach

Testing combines deterministic automated checks, browser interaction, real provider requests and rendered PDF inspection. Each method answers a different question. Unit tests isolate score and validation behaviour; browser tests verify route, form and storage behaviour; provider tests verify actual integration; visual inspection catches output that a presence-only assertion can miss.

Table 5-3: Verification suites and their scope

Suite	Recorded outcome	Scope / qualification
Node automated tests	22 passed, 0 failed	15 backend and 7 frontend regressions.
Initial broad domain suite	74 passed, 7 failed	81 checks before the confirmed fixes; not a final clean run.
Post-fix live checks	17 passed, 0 failed	Seven fixed cases plus selected page/layout/error checks.
Navigation suite	26 local + 26 live passed	Normal motion, lazy downloads, route redirects, tabs and history.
PDF review	Text and rendering checked	Corrected INR amount and readable report layout.
Camera exercise	Controlled QR-video decoded	Exercises media stream and decoder; not physical camera hardware.

Test files and screenshots are retained as evidence. Non-sensitive test inputs protect personal passwords, messages and files during verification. One real counter test was used to verify an increment; bulk tests blocked counter writes to avoid inflating the published number.

5.3.1 Automated regression results

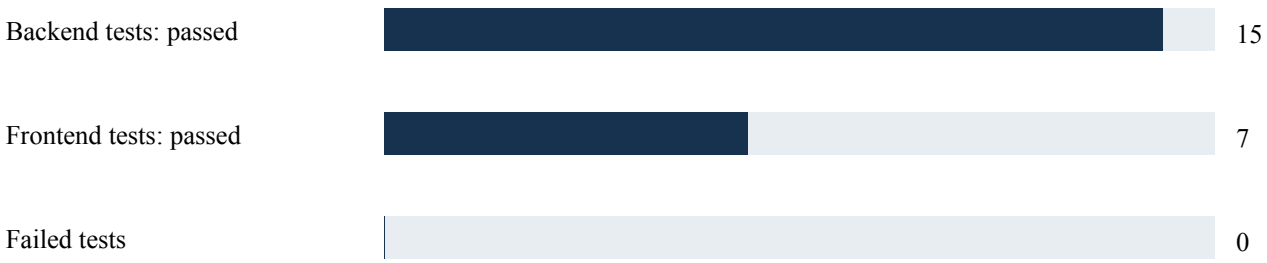


Figure 5-1: Recorded automated test outcomes

Table 5-4: Automated tests by area

Area	Count	Representative coverage
Backend classification	7	OTP negation, URL userinfo, private IPv6, unscored reputation, report wording and stronger local risk.
Backend file scanner	3	Hash mismatch, unscored detection and cached file identity.
CORS origins	2	Local loopback equivalents; production origin is not widened.
VirusTotal statistics	3	Missing, completed and unusable/unfinished statistics.
Frontend IP validation	2	Accepted IPv4/IPv6 forms and malformed-address rejection.
Frontend QR parsing	2	Web links with payment-like parameters; supported UPI schemes.
Frontend URL validation	2	Valid hosts including IPv6; malformed, unsupported and oversized input.
PDF encoding	1	INR summary/breakdown/findings and readable arrow replacement.

Automated tests use the Node test runner and compare actual results with expected outcomes. The recorded pass counts measure functional test coverage. Detection accuracy and false-positive rates require a separate labelled-data evaluation.

5.4 Functional test cases: backend modules

Table 5-5: Functional URL, file and IP cases

ID	Input / action	Expected behaviour	Observed
TC01	Normal Google URL	Show source findings and lower risk	Pass
TC02	Official phishing-test URL	Show Safe Browsing flag; no Safe verdict	Pass
TC03	Malformed URL	Reject before provider request	Pass
TC04	Unsupported FTP scheme	Reject as invalid URL	Pass
TC05	IPv6 URL literal	Accept valid form and reach dispatch	Pass*
TC06	Empty file	Reject in form	Pass
TC07	File above 32 MiB	Reject in form / upload limit	Pass
TC08	Unknown harmless file	Hash lookup, upload and pending report	Pass
TC09	Pending file Check again	Lookup by hash, no repeat byte upload	Pass
TC10	Same hash, renamed file	Preserve current report filename	Pass
TC11	127.0.0.1	Unscored non-public address	Pass
TC12	Expanded/mapped loopback	Same non-public classification	Pass
TC13	Public 8.8.8.8	Fetch reputation and geolocation	Pass
TC14	Malformed 999.1.1.1	Reject as invalid IP	Pass

*The IPv6 form-dispatch regression intercepted its provider request to avoid an unnecessary lookup. IPv6 canonicalisation and non-public reports were also tested against the deployed API. Different test types are identified rather than combined into a single accuracy claim.

5.4.1 Functional cases: messages, QR and passwords

Table 5-6: Browser-local and message tests

ID	Input / action	Expected behaviour	Observed
TC15	KYC/OTP scam text	Dangerous verdict and highlighted phrases	Pass
TC16	Normal meeting message	Lower-risk report with available findings	Pass
TC17	Very short message	Useful form validation	Pass
TC18	Message above 5,000 chars	Backend rejection	Pass
TC19	Generated shop QR image	Decode actual UPI content	Pass
TC20	Controlled QR-camera stream	Media stream yields QR report	Pass
TC21	UPI refund-trap test	Dangerous warning; no payment made	Pass
TC22	Web QR with pa= parameter	Enter URL scanner, not UPI parser	Pass
TC23	Plain-text / Wi-Fi QR	Parse local content and explain it	Pass
TC24	Image without QR	Show no-code-found error	Pass
TC25	Password breach test: password	HIBP match and Dangerous report	Pass
TC26	Password request observation	Five-character prefix; no raw password	Pass
TC27	Refresh a completed result	Retain the latest session report	Pass
TC28	Download UPI report	Readable INR amount in PDF	Pass

The designated test password is widely known and is not associated with a personal account. Camera decoding was verified using a controlled video stream. Device-camera quality and operating-system permission prompts require separate checks.

5.4.2 Defect correction and regression cycle

Table 5-7: Confirmed defects and resulting corrections

Defect	Cause / correction
Web QR misclassified as UPI	Generic pa= detection bypassed link checks; restrict payment schemes.
Late result after leaving scanner	Local result finished before radar delay; check abort before persistence/navigation.
Valid IPv6 URL rejected	Dot-only host rule; align form with valid IP-host support.
Private IPv6 given public score	String spelling bypassed range rules; canonicalise and map IPv4 forms.
Cached filename stale	Whole report reused by hash; overlay the current request metadata.
Skip link produced 404	Hash fragment was interpreted as a route; focus/scroll without route change.
Rupee glyph corrupted in PDF	Built-in font lacked glyph; encode amounts as INR.
Blank page after navigation	Exit animation left new route transparent; render a visible route container.

The defects were reproduced against the published version, corrected in source and verified through targeted checks. Frontend deployment used GitHub Pages; API changes required deploying the updated commit on Render. Provider keys stayed private during publication.

Navigation review identified a route-transition defect that refresh-based checks had not detected. The extended regression suite verifies the visibility of the heading and its ancestor elements in addition to the presence of route content.

5.4.3 Navigation, accessibility and error checks

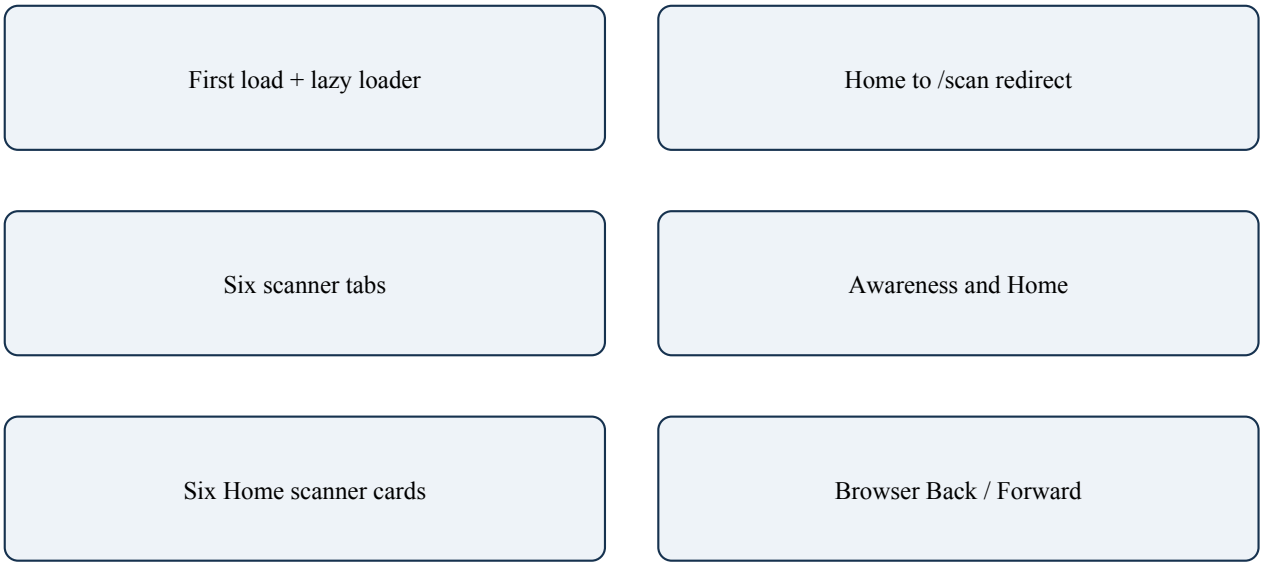


Figure 5-2: Navigation-regression coverage

Twenty-six live navigation checks were completed with normal motion enabled. They cover the Home entry, visible loading during an intentionally slower scanner download, the /scan redirect, six scanner tabs, Awareness, Home scanner cards and browser Back/Forward.

Responsive checks covered Home, all six forms and Awareness at 320, 375 and 768 px. No document-level horizontal overflow was observed in those cases. The mobile menu opens, navigates and closes. Keyboard activation of Skip to content retains the scanner route and moves focus into the main element.

Invalid input produces a local validation message. Controlled network and 429 responses were used to verify that the published UI communicates failure without a blank screen. These two responses were simulated in the test browser and are not claims that the live quota was exhausted.

Reduced-motion and ordinary-motion paths serve different users and must both be tested. Refreshing a route can mask a defect that occurs only when the application navigates between lazy-loaded pages.

5.5 Deployment and configuration

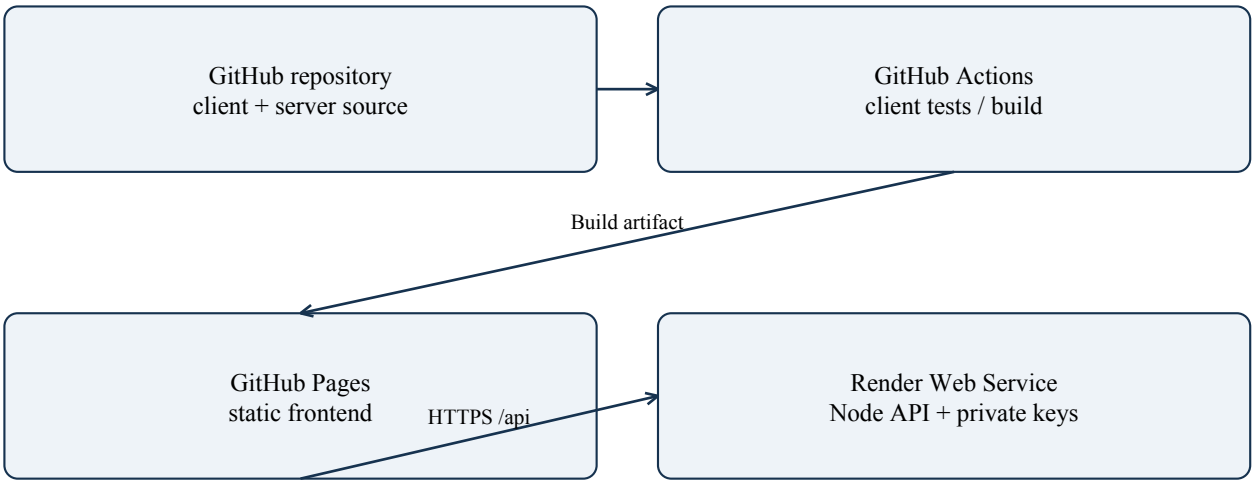


Figure 5-3: GitHub Pages and Render deployment flow

The client workflow installs locked dependencies, runs frontend tests and lint, builds with the /cybershield/ base path and publishes the static assets. VITE_API_BASE_URL is supplied as a repository variable and points to the actual Render URL. Changing that variable requires a rebuild to affect the browser bundle.

The Render Web Service uses the server root directory, npm ci, npm start, Node 22 and /api/health. CLIENT_ORIGIN is https://laxmiyadav72335.github.io, without the /cybershield/ path. Private provider keys are entered as Render environment values.

Table 5-8: Published system addresses

Component	Address
Frontend	https://laxmiyadav72335.github.io/cybershield/
API	https://cybershield-api-r4jc.onrender.com
Source	https://github.com/laxmiyadav72335/cybershield
API health	https://cybershield-api-r4jc.onrender.com/api/health

5.6 Provider limits and verification boundaries

Table 5-9: Relevant provider and application limits

Service	Limit / project guard
VirusTotal public API	4 requests/minute and 500/day; one fresh scan may consume several calls.
Safe Browsing	Project-assigned quota; verify actual allocation in Google Cloud.
Gemini	Model/tier-specific RPM, TPM and daily quotas; shared per project, not per key.
AbuseIPDB Standard	1,000 check requests/day; selected plan may differ.
ip-api free	45 requests/minute per calling IP; HTTP endpoint.
RDAP.org	10 requests in 10 seconds; registry may apply its own limits.
Pwned Passwords	Free hash-range API with no published rate limit.
Backend scanner routes	20 requests per 10 minutes per IP; cached and refresh requests count.
Files / messages / URLs	32 MiB / 5,000 characters / 2,048 characters.
AI explanation / analysis	8-second / 20-second overall caps before fallback.
Render Free	Sleeps after inactivity; first request may need a cold start.

These are documented limits and configured guards, not a measurement of the remaining account allowance. Provider policies can change. VirusTotal public, Safe Browsing and the free ip-api endpoint restrict commercial use. This report concerns an academic application, not an unrestricted commercial deployment.

CHAPTER 6
RESULTS & DISCUSSION

The system provides end-to-end processing through browser-local operations and backend integrations. Inputs are validated, relevant providers are consulted, and findings are returned in a common report. The results below use the recorded test cases and the provider's official phishing test page.

Table 6-1: Recorded functional test results

Case	Score	Verdict	Explanation
Normal Google URL	0/100	Safe	Local / fallback
Official phishing-test URL	95/100	Dangerous	Local / fallback
KYC / OTP scam message	75/100	Dangerous	Local / fallback
UPI refund request	70/100	Dangerous	Local / fallback
Password breach test	100/100	Dangerous	Local / fallback
Private IP 127.0.0.1	Not scored	Unknown	Local / fallback

A Safe result indicates that the consulted sources and local rules did not identify sufficient risk signals for the checked input. A Dangerous result indicates that the scam-message or breached-password test met the configured risk conditions. Results should be interpreted together with source availability and timestamps.

The private-IP result is intentionally unscored. A pending file report is also unscored until VirusTotal supplies usable detection statistics. Preserving these states is preferable to displaying a reassuring score based on no completed check.

6.1 Interpretation of observed results

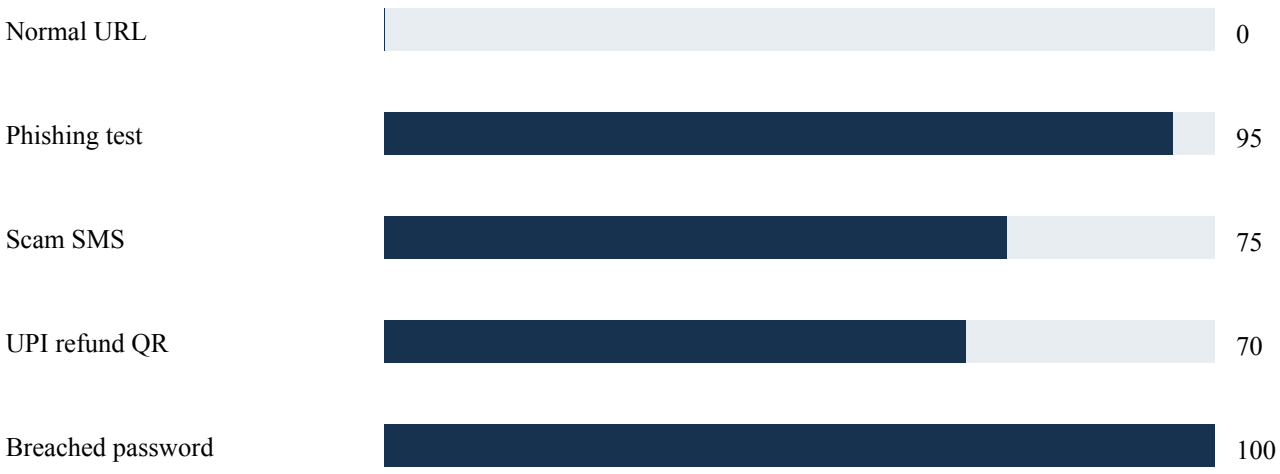


Figure 6-1: Scores of executed functional test cases

The recorded cases show the influence of different evidence sources. The official phishing test URL is supported by a Safe Browsing match and antivirus flags. The scam SMS combines AI assessment with deterministic patterns. The UPI refund QR is scored locally from the encoded payment context. The password result is driven by an actual HIBP hash-range match.

The results document functional behaviour for the executed cases. Quantitative detection evaluation requires independently labelled records, a documented data-selection method and consistent experimental conditions. Precision, recall and a confusion matrix are outside the present evaluation scope.

Provider evidence is time-dependent and may change after caching, new reports or later scans. AI availability can also differ between requests. The project records these conditions through timestamps, source cards, cache labels and partial-result notices.

6.2 Screenshots: Home and Link Scanner

The Home page presents the six scanner choices and a live counter. The scanner page provides a consistent form area, input assistance and source list. These captures are from the deployed CyberShield frontend.

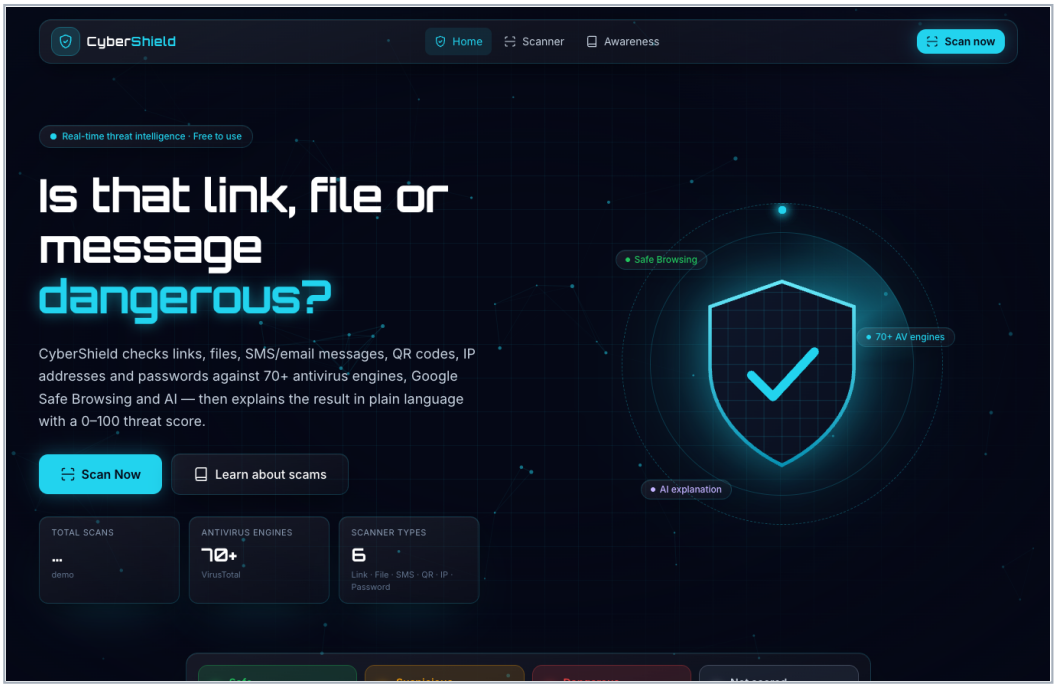


Figure 6-2: CyberShield Home page

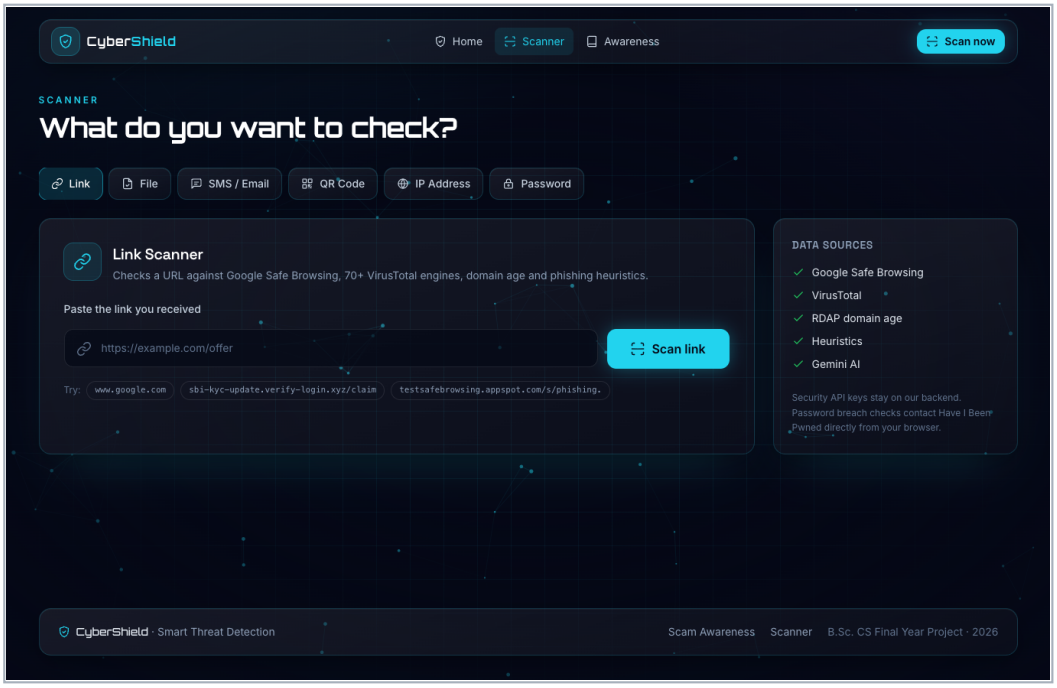


Figure 6-3: Link Scanner form and source panel

6.2.1 Screenshots: File and SMS / Email

The File form describes hash-first processing and the size limit. The message form provides input assistance, a character counter and clear submission controls.

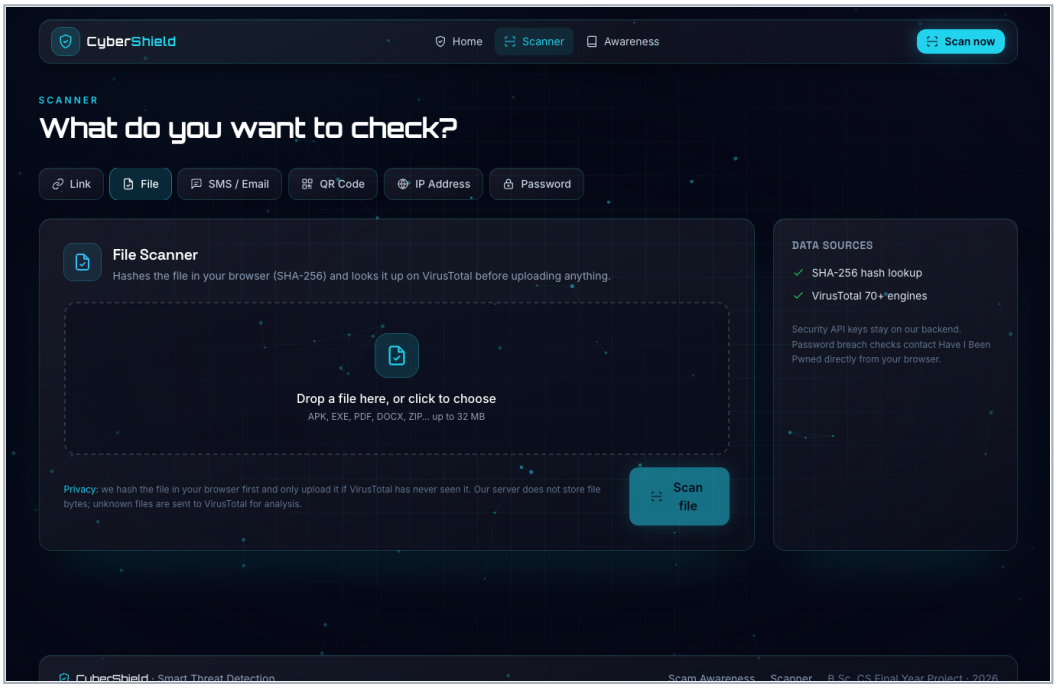


Figure 6-4: File Scanner input

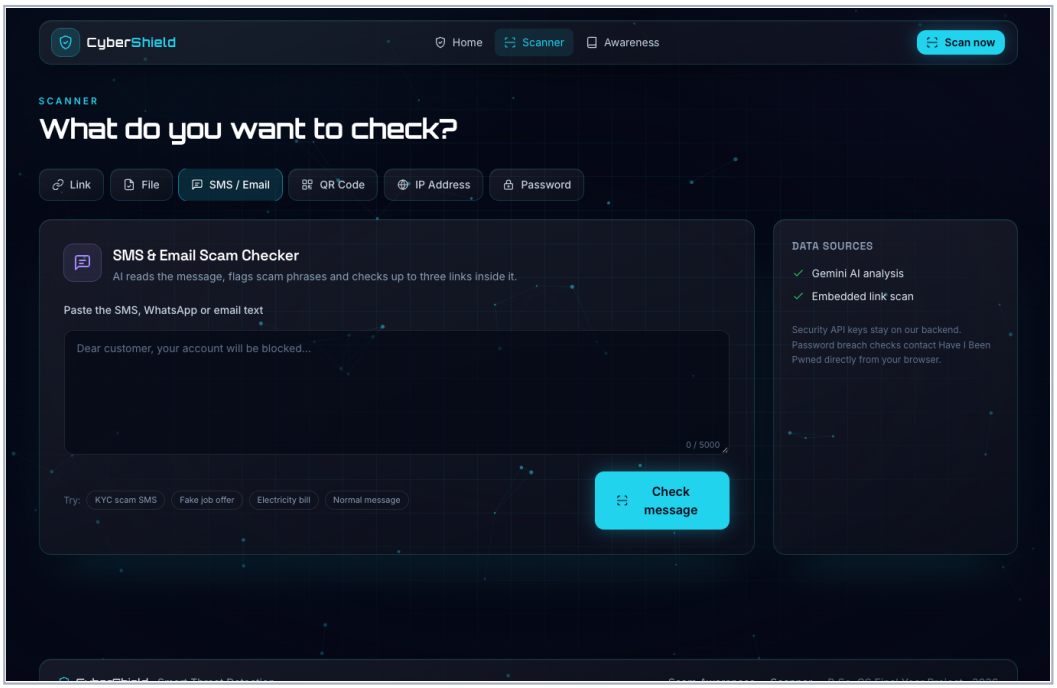


Figure 6-5: SMS / Email Scanner input

6.2.2 Screenshots: QR and IP

The QR module offers image, camera and pasted-text modes. The IP module accepts both IPv4 and IPv6 and explains its reputation source.

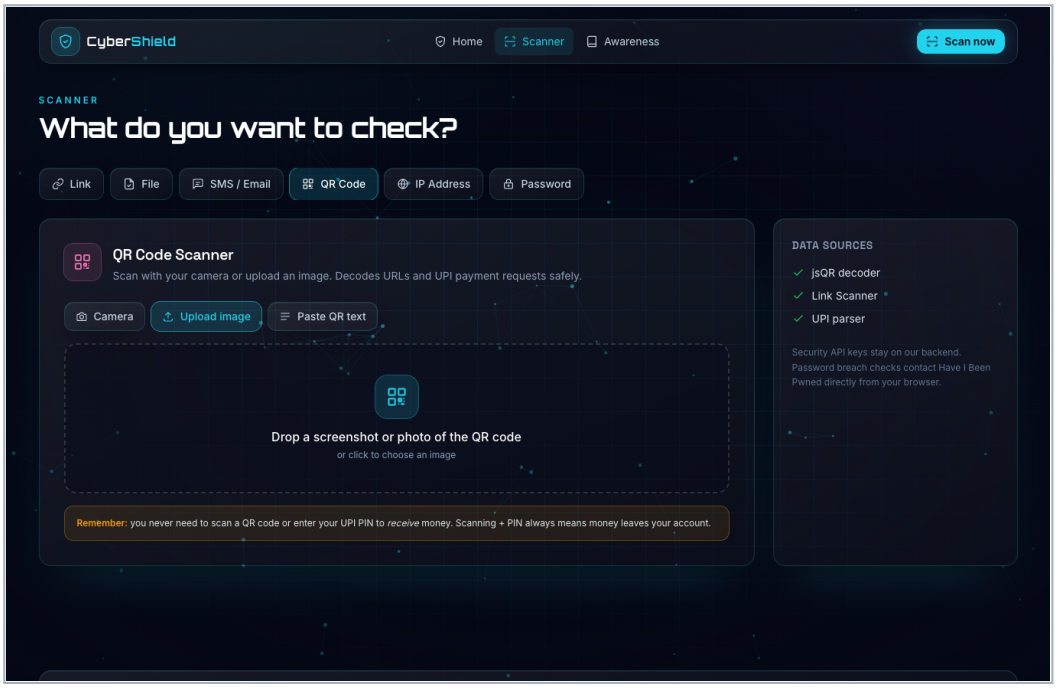


Figure 6-6: QR Scanner input methods

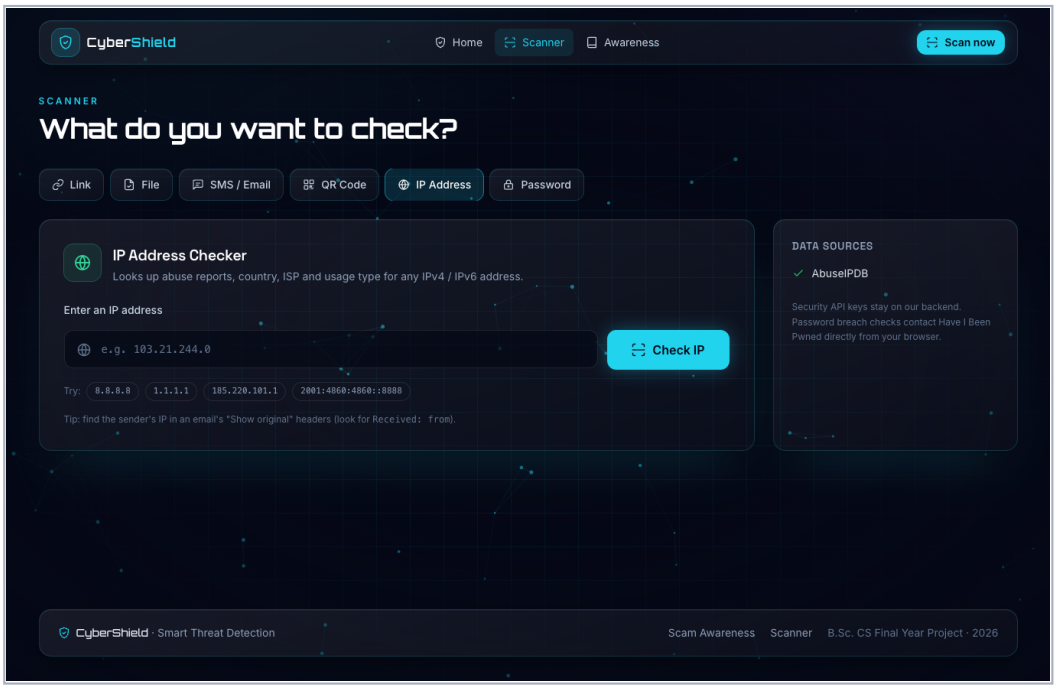


Figure 6-7: IP Address Checker form

6.2.3 Screenshots: Password and Awareness

Password processing is browser-local except for the hash-prefix request. Awareness content explains common Indian scam scenarios and encourages independent verification.

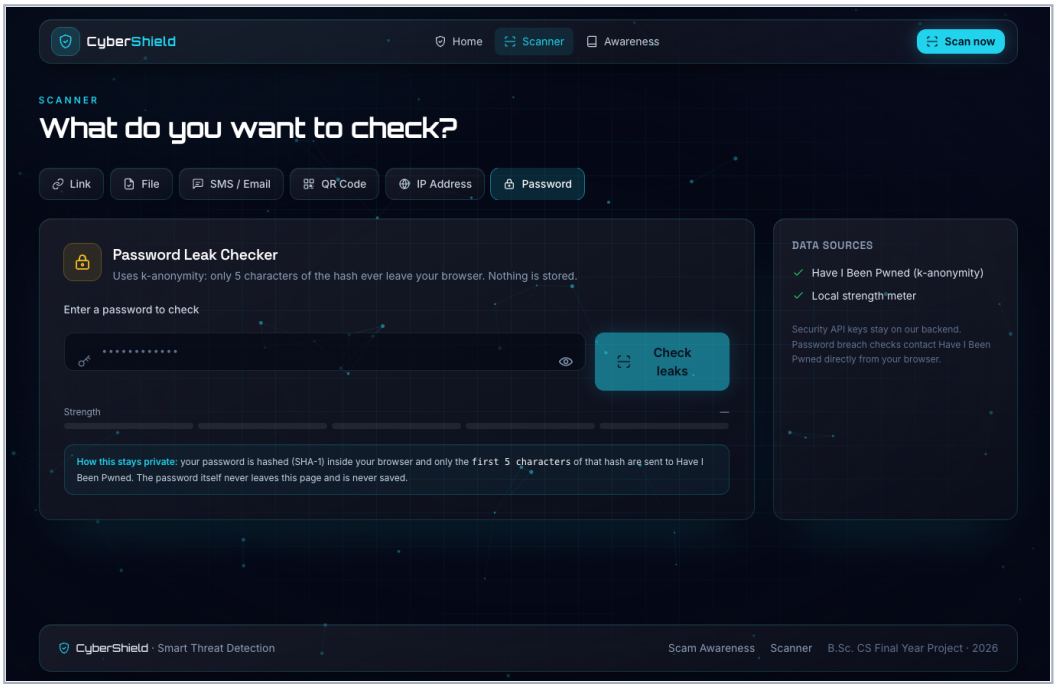


Figure 6-8: Password Leak Checker and privacy explanation

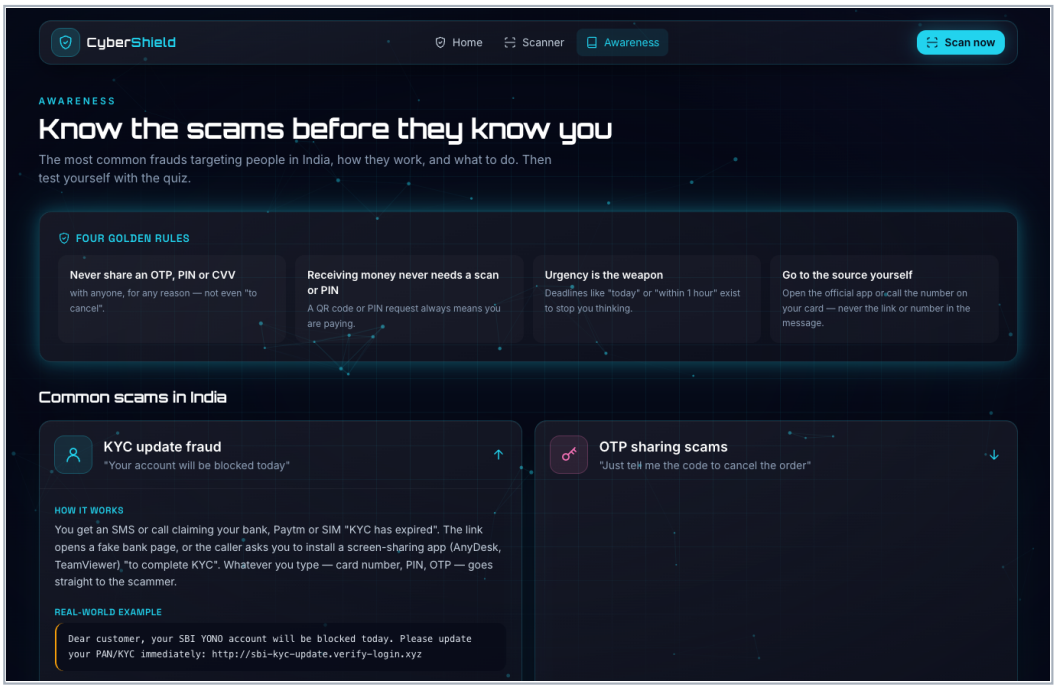


Figure 6-9: Awareness page and golden rules

6.2.4 Screenshots: Ordinary and flagged URLs

The Result page presents findings for both an ordinary website and the provider's official phishing test page. The phishing case verifies that the system displays the provider flag and the associated warning verdict.

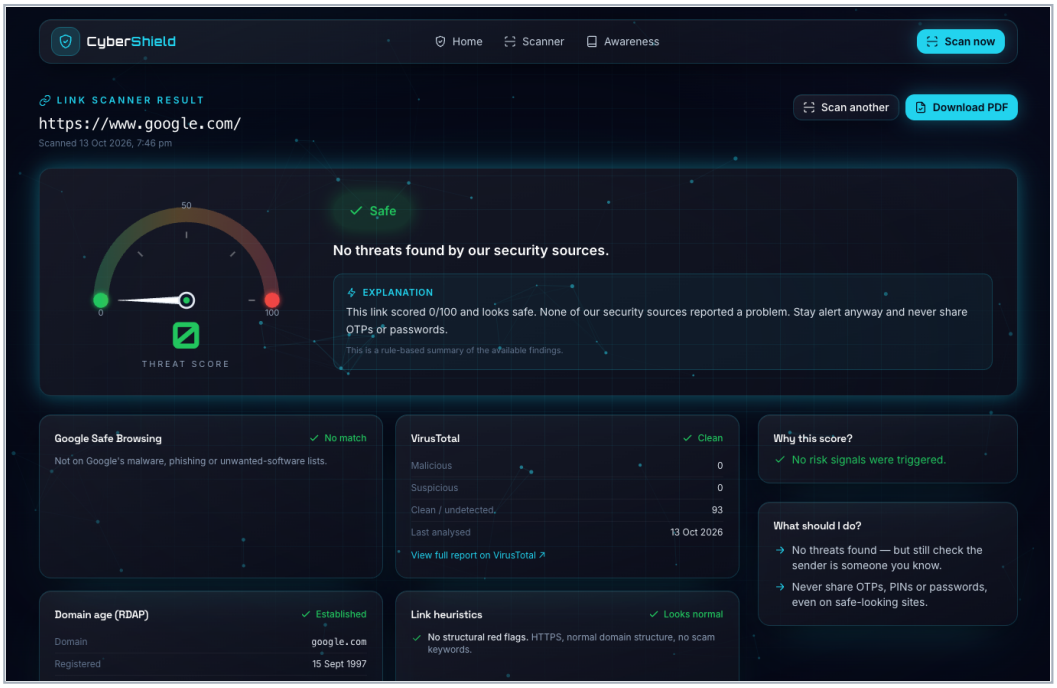


Figure 6-10: Report for the normal Google URL

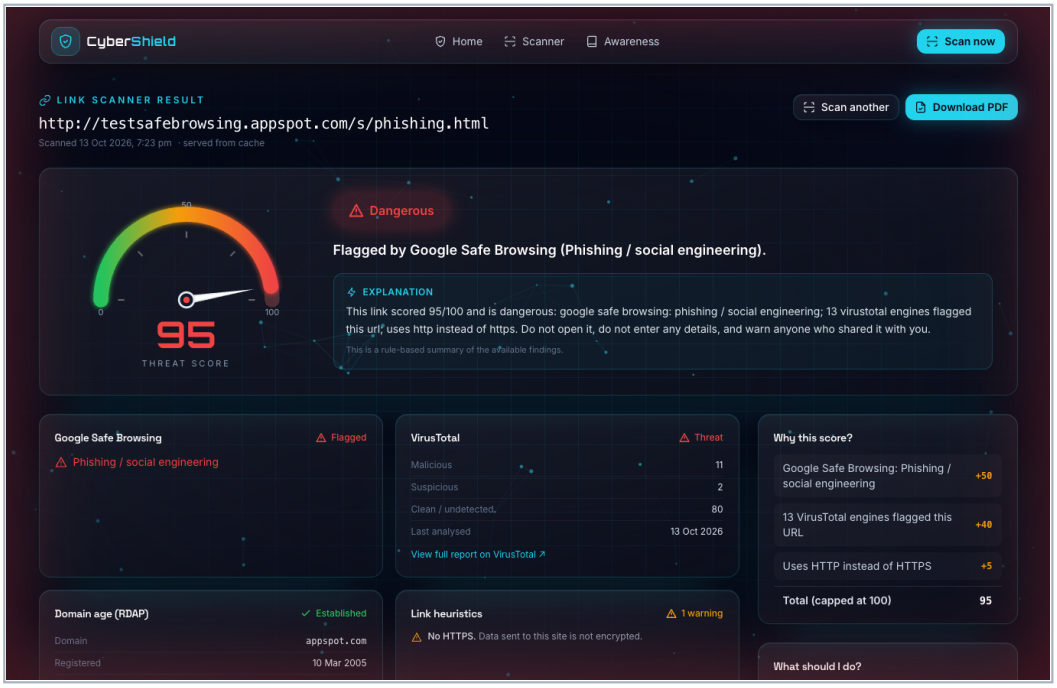


Figure 6-11: Report for the official phishing test URL

6.2.5 Screenshots: UPI and scam-message results

The UPI test input contains a pre-filled amount and refund-related wording. The scam-message test requests KYC action and OTP disclosure. Evaluation records the findings without making a payment or disclosing account credentials.

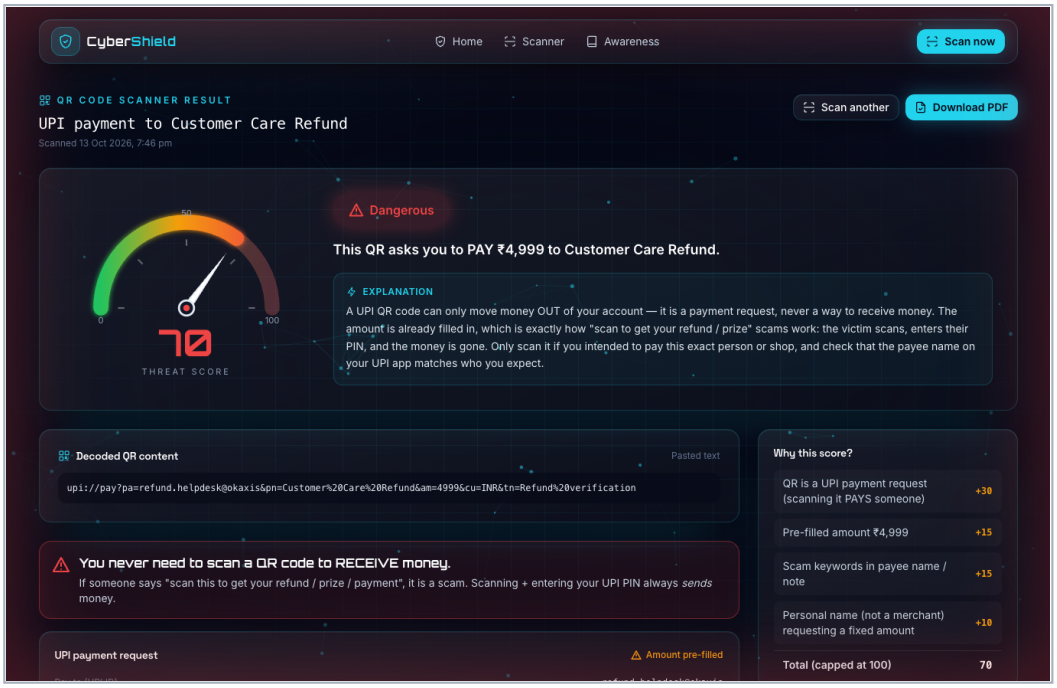


Figure 6-12: Dangerous UPI refund-trap report

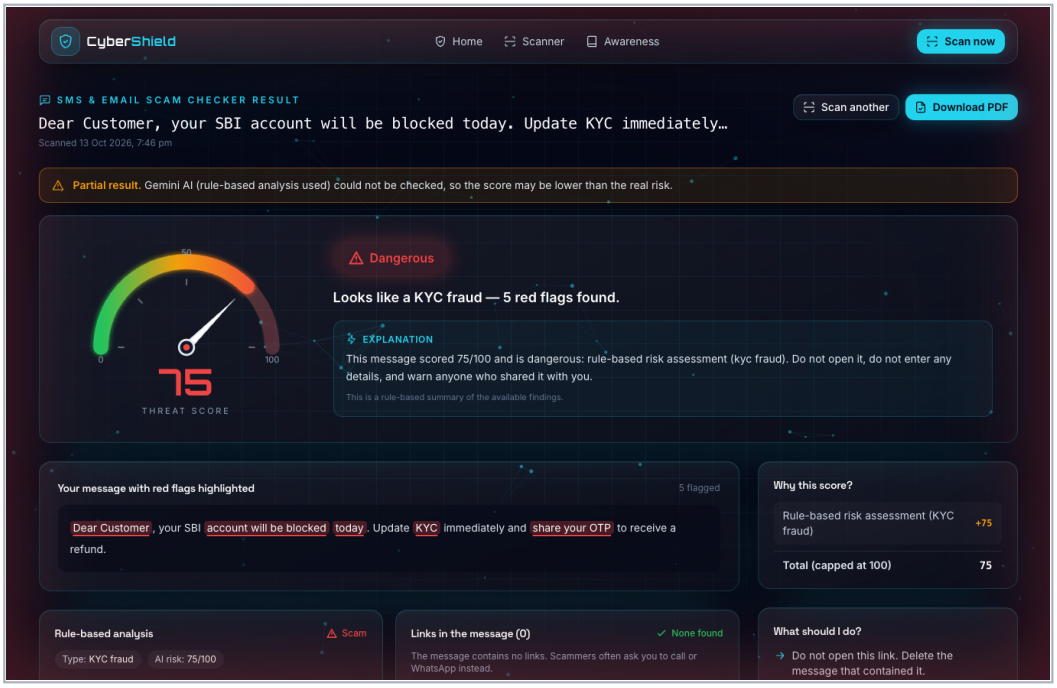


Figure 6-13: Scam SMS report and explanation

6.2.6 Screenshots: Private IP and password exposure

A private address has no applicable public reputation score. The password-exposure test uses the designated input password; the saved report contains a masked preview and excludes the raw password.

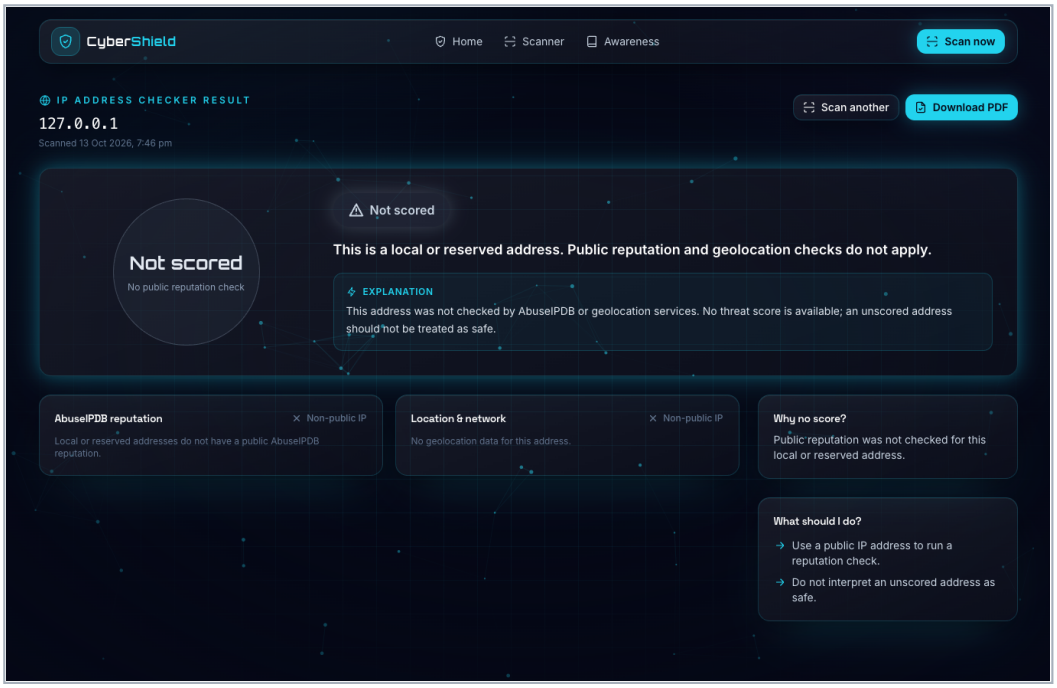


Figure 6-14: Unscored private-IP report

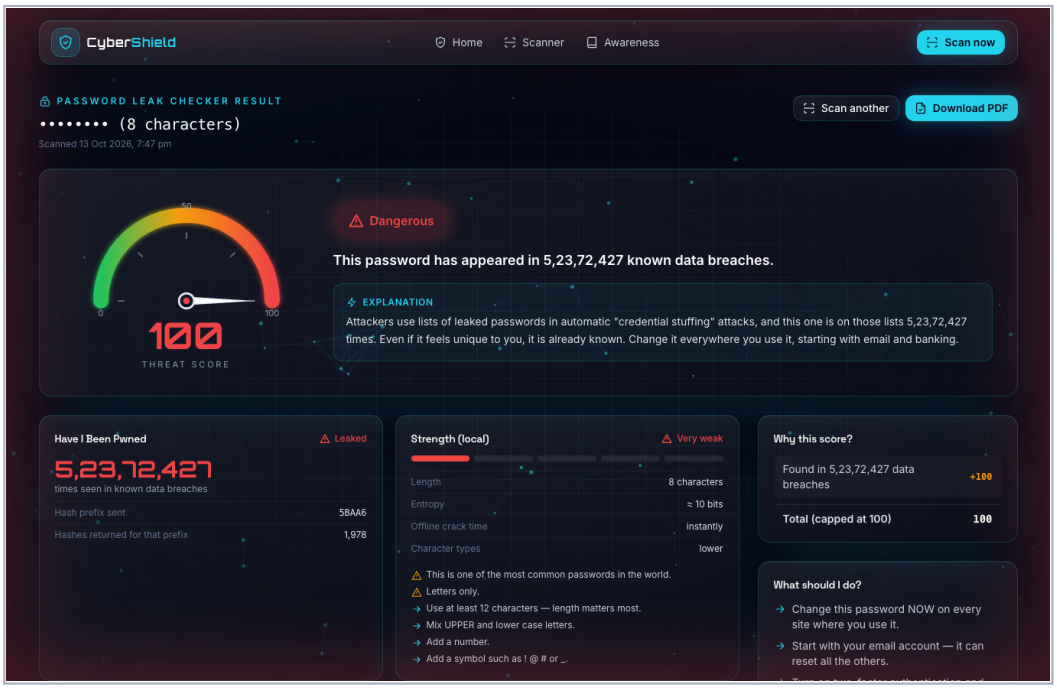


Figure 6-15: Breached-password report

6.2.7 Screenshots: Quiz and exported PDF

The quiz gives immediate feedback and retains a best score in the browser. PDF export summarises the input, score, findings and recommended action. INR text avoids the unsupported rupee glyph in the standard PDF font.

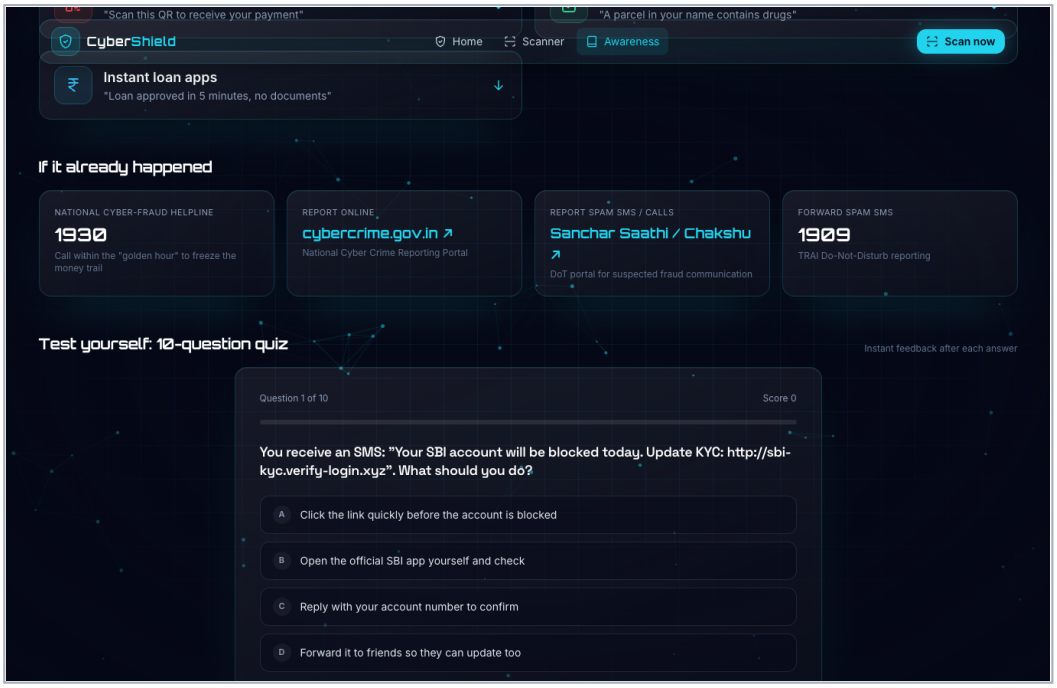


Figure 6-16: Ten-question awareness quiz

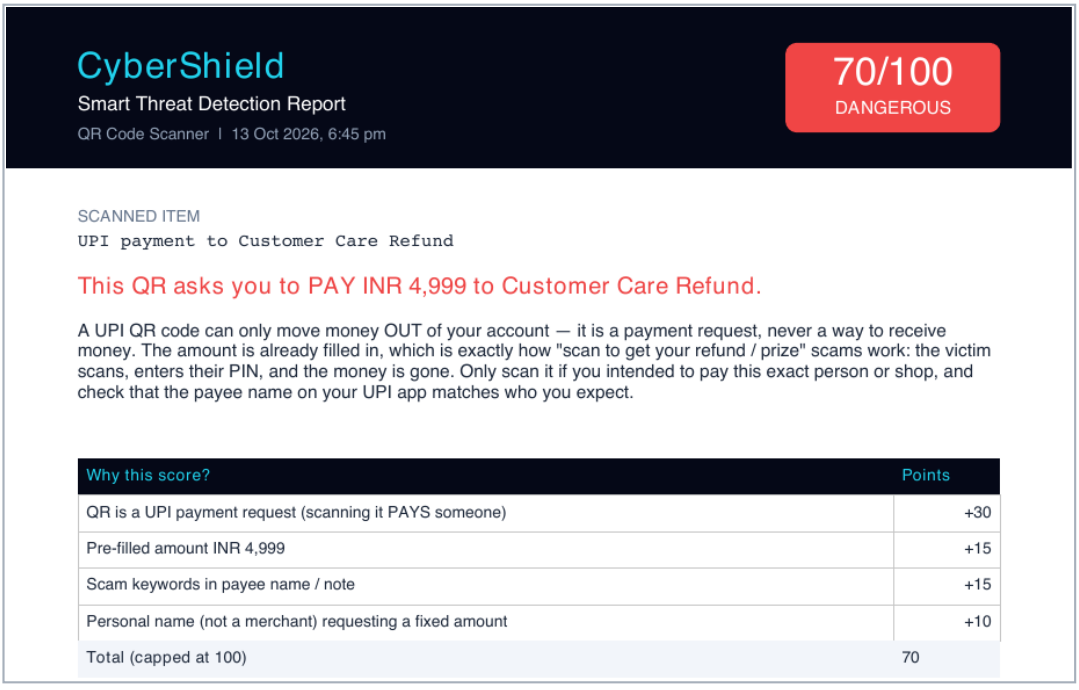


Figure 6-17: Exported UPI report excerpt with readable currency

6.2.8 Screenshots: Repository and API deployment

The public repository contains the client, API and deployment configuration. The health endpoint confirms the API is reachable and indicates whether each provider key is configured, without revealing the key values.

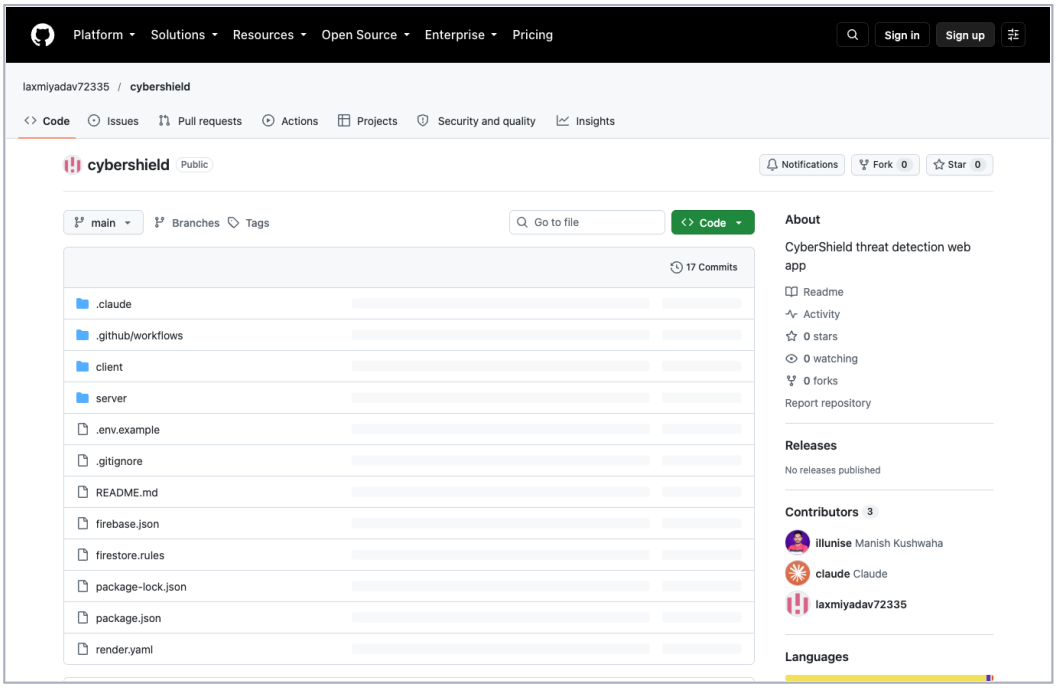


Figure 6-18: Published CyberShield repository

GET https://cybershield-api-r4jc.onrender.com/api/health


```
{
  "ok": true,
  "service": "cybershield-api",
  "time": "2026-10-13T14:39:04.685Z",
  "apis": {
    "virusTotal": true,
    "safeBrowsing": true,
    "gemini": true,
    "abuseIpdb": true
  }
}
```


Figure 6-19: Formatted response from the deployed API health endpoint

6.3 Advantages and limitations

6.3.1 Advantages

• One interface combines six complementary security checks.

• The score breakdown and source cards make findings inspectable.

• Hash-first file lookup avoids unnecessary byte uploads for known files.

• HIBP range lookup keeps raw passwords inside the browser.

• UPI QR fields are exposed before the user considers any payment.

• Rule fallback provides useful message analysis when AI is unavailable.

• Reports survive refresh and can be downloaded for later reference.

• The Awareness section supports practical cyber-security learning.

6.3.2 Limitations

• Unknown or newly emerging threats may not appear in provider data.

• Weighted scores are not calibrated probabilities and can produce false positives or negatives.

• A single provider outage or quota limit reduces the available evidence.

• Provider findings, domain data and cached results can become stale.

• A public scan counter can be manipulated despite its restricted update shape.

• Messages may be sent to Gemini and unknown files to VirusTotal; session storage may retain report content.

• The standard PDF fonts do not cover every writing system.

• Physical camera behaviour depends on device, lighting and permissions.

• A free Render cold start can delay backend-backed scans.

6.4 Future enhancements

• Add a shared provider request queue and quota-aware backoff, particularly for VirusTotal.

• Use persistent caching with clear freshness controls across API restarts.

• Calibrate scoring thresholds with a representative labelled evaluation dataset.

• Raise authoritative positive threat evidence to an appropriate severity floor after evaluation.

• Offer multilingual explanations and embed fonts for Indian-language PDF text.

• Add explicit consent or clearer controls around external file and message submission.

• Make report persistence configurable and provide a clear delete-report action.

• Introduce optional authenticated history with retention rules if the scope requires it.

• Improve QR parser handling for escaped Wi-Fi fields and additional validated payment formats.

• Broaden browser/device and physical-camera accessibility testing.

• Monitor backend errors, provider availability and cold-start behaviour.

• Optimise the main frontend bundle and defer nonessential dependencies.

These suggestions are development opportunities rather than implemented claims. Any extension involving accounts or stored scan history requires additional data minimisation, access control and deletion design. A future statistical evaluation must distinguish provider truth labels from the project's own score output.

The immediate engineering principle is to preserve the transparency of the existing design: display what was checked, what was unavailable, how the score was formed and what a user can do next.

CHAPTER 7
CONCLUSION

CyberShield - Smart Threat Detection was developed as an integrated educational application for reviewing common digital-security risks. It brings together links, files, SMS/email messages, QR codes, IP addresses and password breach exposure within a consistent browser interface.

The implementation combines React component design, API orchestration with Node and Express, browser cryptography, local QR decoding, deterministic scoring, Gemini-based explanations, session persistence and PDF generation. Private security-provider keys remain in the backend environment. The public Firestore counter is separated from report content, and raw passwords are not transmitted to the application API.

Testing identified and corrected practical correctness and usability defects, including misclassified web QR codes, late navigation after cancellation, alternate private-IP spellings, stale cached filenames, invalid keyboard routing, corrupted currency glyphs and a blank route caused by animation state. The process showed why normal user navigation, visibility assertions and visual export inspection matter alongside automated function tests.

The recorded automated and live checks verify the implemented workflows for the executed cases. Detection coverage depends on provider intelligence and configured heuristics, while browser behaviour depends on the target device. Findings are interpreted with their availability, freshness and scope in mind.

The project fulfils its practical objective of making security evidence understandable and accessible. Further work can improve quota handling, score calibration, multilingual support, operational reliability and evaluation coverage. CyberShield applies modern web engineering to practical cyber-awareness and risk assessment.

CHAPTER 8
REFERENCES

1. React documentation. https://react.dev/

2. Vite guide and build configuration. https://vite.dev/guide/

3. React Router documentation. https://reactrouter.com/

4. Tailwind CSS documentation. https://tailwindcss.com/docs

5. Motion for React documentation. https://motion.dev/docs/react

6. Node.js and test runner documentation. https://nodejs.org/api/test.html

7. Express documentation. https://expressjs.com/

8. VirusTotal API v3 and public API limits. https://docs.virustotal.com/reference/public-vs-premium-api

9. Google Safe Browsing v4. https://developers.google.com/safe-browsing/v4

10. Gemini API documentation and rate limits. https://ai.google.dev/gemini-api/docs/rate-limits

11. AbuseIPDB API v2 documentation. https://docs.abuseipdb.com/

12. ip-api JSON documentation. https://ip-api.com/docs/api:json

13. RDAP.org service and usage. https://about.rdap.org/

14. HIBP Pwned Passwords API. https://haveibeenpwned.com/API/v3

15. Firebase Firestore documentation. https://firebase.google.com/docs/firestore

16. jsQR project. https://github.com/cozmo/jsQR

17. jsPDF and AutoTable. https://github.com/parallax/jsPDF

18. GitHub Pages documentation. https://docs.github.com/en/pages

19. Render free-service documentation. https://render.com/docs/free

20. CyberShield implementation repository. https://github.com/laxmiyadav72335/cybershield

Documentation consulted and implementation/test evidence reviewed on 13 October 2026. Source-code excerpts and screenshots in this report describe the CyberShield project. Provider terms and quotas should be checked again when the deployment or account plan changes.